Advanced Functional Programming with Elixir

Chapter 1 - Build FunPark: Model Real-World Data

- We are going to model a theme park using Domain-Driven Design to divide our system into *bounded contexts*. This way each context owns part of the domain and defines how its data and behavior fit together. This will allow us to apply functional techniques within well-defined boundaries, ensuring that change stays local and models stay aligned with real-world concerns. We can split FunPark into Rides, Patrons, and FastPass.
- In bounded contexts, names matter. Terms should match the language used by the people who work in and understand that part of the domain.
- An *ubiquitous language* is shared vocabulary used consistently by developers, designers, and domain experts within a bounded context.
- Here are our main entities in this book and their `make` factory methods:

```elixir
# lib/fun_park/ride.ex
defmodule FunPark.Ride do
  defstruct id: nil,
            name: "Unknown Ride",
            min_age: 0,
            min_height: 0,
            wait_time: 0,
            online: true,
            tags: []

  def make(name, opts \\ []) when is_binary(name) do
    %__MODULE__{
      id: :erlang.unique_integer([:positive]),
      name: name,
      min_age: Keyword.get(opts, :min_age, 0),
      min_height: Keyword.get(opts, :min_height, 0),
      wait_time: Keyword.get(opts, :wait_time, 0),
      online: Keyword.get(opts, :online, true),
      tags: Keyword.get(opts, :tags, [])
    }
  end
end
```

- `defstruct` doesn't take a name because it always attaches to the enclosing module — the module name is the struct's name, and a module can define at most one struct. `__MODULE__` is a compile-time macro that expands to that enclosing module's name, so `%__MODULE__{}` is just shorthand for `%FunPark.Ride{}` written inside `FunPark.Ride`. If you renamed the module to Attraction in the `defmodule` line, every `%__MODULE__{}` reference inside that file — including the one in `make/2` — would automatically resolve to `%Attraction{}` with no other edits needed.

```elixir
# lib/fun_park/fast_pass.ex
defmodule FunPark.FastPass do

  defstruct id: nil,
            ride: nil,
            time: nil

  def make(%Ride{} = ride, %DateTime{} = time) do
    %__MODULE__{
      id: :erlang.unique_integer([:positive]),
      ride: ride,
      time: time
    }
  end
end

iex> mansion = FunPark.Ride.make("Dark Mansion", min_age: 14, tags: [:dark])
# This one requires a DateTime, so to use it we might:
iex> datetime = DateTime.new!(~D[2025-06-01], ~T[13:00:00])
iex> fast_pass = FunPark.FastPass.make(mansion, datetime)
```

- A note on constructor names.
  - `new` (no bang) — construction always succeeds, or returns `{:ok, x} | {:error, reason}` so the caller has to handle failure explicitly.
  - `new!` (bang) — construction can fail, and on failure it raises instead of returning an error tuple. The ! is a hard convention signal meaning "this will raise — expect an exception if things go wrong, not a return value you need to pattern-match."
  - `make`, `build`, `from_x` — no real convention weight; these are just "whatever the author felt like." You'll see all three used interchangeably for the same non-failing-constructor purpose.

```elixir
# lib/fun_park/patron.ex
defmodule FunPark.Patron do
  defstruct id: nil,
            name: nil,
            age: 0,
            height: 0,
            ticket_tier: :basic,
            fast_passes: [],
            reward_points: 0,
            likes: [],
            dislikes: []

  def make(name, age, height, opts \\ [])
      when is_binary(name) and
             is_integer(age) and
             is_integer(height) and
             age > 0 and
             height > 0 do
    %__MODULE__{
      id: :erlang.unique_integer([:positive]),
      name: name,
      age: age,
      height: height,
      ticket_tier: Keyword.get(opts, :ticket_tier, :basic),
      fast_passes: Keyword.get(opts, :fast_passes, []),
      reward_points: Keyword.get(opts, :reward_points, 0),
      likes: Keyword.get(opts, :likes, []),
      dislikes: Keyword.get(opts, :dislikes, [])
    }
  end
end
```

Chapter 2 - Implement Domain-Specific Equality with Protocols

- Equality is reflexive (a = a), symmetric (if a = b, then b = a), and transitive (if a = b and b = c, then a = c).
- Equality is always defined within a bounded context. There is no meaningful comparison between a Ride and a Patron.
- Equality should be polymorphic, able to adapt to the kind of input it compares. In Elixir, the struct tag (the module name) becomes this dispatch key for protocols, allowing

polymorphic behavior through dynamic dispatch. This allows each struct to define what equality means in its own context.

```
# lib/fun_park/eq.ex
defprotocol FunPark.Eq do
  @fallback_to_any true

  def eq?(a, b)

  def not_eq?(a, b)
end

defimpl FunPark.Eq, for: Any do
  def eq?(a, b), do: a == b
  def not_eq?(a, b), do: a != b
end
```

- We define the `Eq` protocol and the fallback implementation for `Any` in eq.ex, but we define the implementations for the structs in their module files.
- We include both `eq?` and `not_eq?` because in some cases inequality may be more efficient than equality.
- Protocols in Elixir are resolved at runtime, so if we attempt to compare to values that lack an `Eq` implementation, the call raises a runtime error. If a type doesn't explicitly implement `Eq`, Elixir falls back to a generic implementation for `Any`, since we specified `@fallback_to_any true`. We have to write this implementation for `Any` like we did above.
- Before we define `Eq`, let's introduce a way to change our structs:

```
# lib/fun_park/patron.ex
def change(%__MODULE__{} = patron, attrs) when is_map(attrs) do
  attrs = Map.delete(attrs, :id)

  struct(patron, attrs)
end
```

- `struct/2` updates an existing struct by merging a map of new values. Only keys that match the structs defined field are applied and extras are ignored. Here we delete the `id` key to prevent changing the entity's identity.

- We say `change/2` is *closed*. We refer to *closure*, a function is *closed* if it returns the same type it receives. When each step in a pipeline stays within the same type, it becomes easy to chain transformations without extra wrappers, branching, or conversion logic.
- Our current definition of `Eq` for `Any` doesn't work in our bounded context of Patrons. If a Patron changes its ticket_tier, it currently is considered a different patron. We want to consider two Patrons as equal if they share the same `id`, because that is their identity.

```
defimpl FunPark.Eq, for: FunPark.Patron do
  alias FunPark.Eq
  alias FunPark.Patron

  def eq?(%Patron{id: v1}, %Patron{id: v2}), do: Eq.eq?(v1, v2)
  def not_eq?(%Patron{id: v1}, %Patron{id: v2}), do: Eq.not_eq?(v1, v2)
end
```

- `Eq` uses *projection*, where a value is transformed - typically by extracting a field - so equality can be defined based on that simpler representation. This allows us to reuse existing comparisons on more complex structures.
- We define `change/2` and `Eq` similarily for `Ride` and `FastPass`. It is tempting to define a shared `change` utility function, but this would undermine the separation between contexts. Each entity could change in a different way.
- Our domain expert already brought up a problem. For a Fast Pass two with the same Datetime are effectively duplicates because the Patron cannot be in two places at the same time. But we have a problem. We still want to consider two Fast Passes with the same `id` equal. We are going to deal with this by defining a `contramap` function. It is built in for many functional languages, but not Elixir. Functional programming includes the *contravariant functor*, which transforms the input before its processed. This is used for abstractions like `Eq` and `Ord` where we want to transform a value before comparison.

```elixir
# lib/fun_park/eq/utils.ex
defmodule FunPark.Eq.Utils do
  alias FunPark.Eq

  def contramap(f) do
    %{
      eq?: fn a, b -> Eq.eq?(f.(a), f.(b)) end,
      not_eq?: fn a, b -> Eq.not_eq?(f.(a), f.(b)) end
    }
  end
end
```

- `contramap/1` is a *higher-order transformer*. It takes an existing comparator and adapts it to work on a different type by applying a function *before* comparing values.
- `contramap/1` is similar to `map` which loosely speaking "transforms values in a container", but it's less 'transform, then compare' and more 'compare using a temporary view of the data, computed on the fly.
- We're not actually going to change our `Eq` definition from identity. We're just going to add a new `eq_time` function that compares time.

```elixir
# lib/fun_park/fast_pass.ex
def get_ride(%__MODULE__{ride: ride}), do: ride

def eq_ride do
  Eq.Utils.contramap(&get_ride/1)
end

def get_time(%__MODULE__{time: time}), do: time

def eq_time do
  Eq.Utils.contramap(&get_time/1)
end
```

- Our new `contramap` map of equality functions is different from our protocol tied to named modules, so we're going to normalize everything to maps. That way `eq?` and `not_eq?` are always map keys. Then we improve `contramap/1`.

```elixir
# lib/fun_park/eq/utils.ex
def to_eq_map(%{eq?: eq_fun, not_eq?: not_eq_fun} = eq_map)
    when is_function(eq_fun, 2) and is_function(not_eq_fun, 2) do
  eq_map
end

def to_eq_map(module) when is_atom(module) do
  %{
    eq?: &module.eq?/2,
    not_eq?: &module.not_eq?/2
  }
end

def contramap(f, eq \\ Eq) do
  eq = to_eq_map(eq)

  %{
    eq?: fn a, b -> eq.eq?.(f.(a), f.(b)) end,
    not_eq?: fn a, b -> eq.not_eq?.(f.(a), f.(b)) end
  }
end
```

- So now we've standardized the representation and `eq \\ Eq` preserves the default while still allowing us to swap in custom equality logic.

```elixir
# lib/fun_park/eq/utils.ex
def eq?(a, b, eq \\ Eq) do
  eq = to_eq_map(eq)
  eq.eq?.(a, b)
end
```

- When we wrap `Eq`, calls still dispatch through the protocol exactly as before, including the fallback to `Any`.
- Now callers have a single entry point that uses the protocol by default but still allows custom equality logic to be passed in as an optional parameter.

```
iex> mansion = FunPark.Ride.make("Dark Mansion", min_age: 14, tags: [:dark])
iex> tea_cup = FunPark.Ride.make("Tea Cup")
iex> datetime = DateTime.new!(~D[2025-06-01], ~T[13:00:00])
iex> fast_pass_a = FunPark.FastPass.make(mansion, datetime)
iex> fast_pass_b = FunPark.FastPass.make(tea_cup, datetime)
# different id's make different passes in the context of FastPass
iex> FunPark.Eq.Utils.eq?(fast_pass_a, fast_pass_b) # false
iex> has_eq_time = FunPark.FastPass.eq_time()
iex> FunPark.Eq.Utils.eq?(fast_pass_a, fast_pass_b, has_eq_time) # true
```

- So now we have the problem of equality solved for our domain, we need to implement a number of list operations that respect it.

```elixir
# lib/fun_park/list.ex
# In production grade code, we'd probably track uniqueness with a hashmap.
# Despite cool list reversing tricks, this algorithm is O(n^2) when it
# could be O(n).
def uniq(list, eq \\ FunPark.Eq) when is_list(list) do
  list
  |> Enum.reduce([], fn item, acc ->
    if Enum.any?(acc, &Eq.Utils.eq?(item, &1, eq))
      do: acc,
      else: [item | acc]
  end)
  |> :lists.reverse()
end

def union(list1, list2, eq \\ FunPark.Eq)
    when is_list(list1) and is_list(list2) do
  (list1 ++ list2) |> uniq(eq)
end

def intersection(list1, list2, eq \\ FunPark.Eq)
    when is_list(list1) and is_list(list2) do
  list1
  |> Enum.filter(fn item ->
    Enum.any?(list2, &Eq.Utils.eq?(item, &1, eq))
  end)
  |> uniq(eq)
end

def difference(list1, list2, eq \\ FunPark.Eq)
    when is_list(list1) and is_list(list2) do
  list1
  |> Enum.reject(fn item ->
    Enum.any?(list2, &Eq.Utils.eq?(item, &1, eq))
  end)
  |> uniq(eq)
end

def symmetric_difference(list1, list2, eq \\ FunPark.Eq)
    when is_list(list1) and is_list(list2) do
  (difference(list1, list2, eq) ++
    difference(list2, list1, eq))
  |> uniq(eq)
end

def subset?(small, large, eq \\ FunPark.Eq)
```

```
    when is_list(small) and is_list(large) do
  Enum.all?(small, fn item ->
    Enum.any?(large, &Eq.Utils.eq?(item, &1, eq))
  end)
end

def superset?(large, small, eq \\ FunPark.Eq)
    when is_list(small) and is_list(large) do
  subset?(small, large, eq)
end
```

- Note above how we use `:` to call native Erlang function `:lists.reverse()`. This is idiomatic Elixir for native functions, but we didn't really need it here since Elixir provides a `Enum.reverse/1` wrapper.
- A good exercise is a `contramap` that normalizes strings before comparing equality:

```
iex> string_1 = "Alice"
iex> string_2 = "  alice "
iex> FunPark.Eq.Utils.eq?(string_1, string_2) # false
iex> normalize = fn s -> s |> String.trim() |> String.downcase() end
iex> eq_normalized = FunPark.Eq.Utils.contramap(normalize)
iex> FunPark.Eq.Utils.eq?(string_1, string_2, eq_normalized) # true
# we could easily have other comparison function, so this is very flexible.
iex> by_id = contramap(& &1.id)
iex> by_lowercase_name = contramap(& &1.name |> String.downcase())
```

- Equality isn't just a basic comparison - it reflects the rules of the domain. For patrons, rides, or FastPasses, *equal* means what the business says it means. The big shift in functional programming is moving from ad hoc checks scattered around the codebase to a composable structure that stays flexible as the domain evolves.

Chapter 3 - Create Flexible Ordering with Protocols

- Ordering builds on equality. If we can tell which of two things comes first, we can also tell if they're equal. In a typed language, we would say `Ord` extends `Eq`, but in Elixir protocols are independent.
- Ordering is reflexive (a <= a), antisymmetric (a <= b and b <=a -> a = b), and transitive (a <= b and b <= c -> a <= c).

```elixir
# lib/fun_park/ord.ex
defprotocol FunPark.Ord do
  @fallback_to_any true

  def lt?(a, b)
  def le?(a, b)
  def gt?(a, b)
  def ge?(a, b)
end

defimpl FunPark.Ord, for: Any do
  def lt?(a, b), do: a < b
  def le?(a, b), do: a <= b
  def gt?(a, b), do: a > b
  def ge?(a, b), do: a >= b
end
```

- Elixir doesn't allow default implementations for protocols, but specifying `@fallback_to_any` and defining an implementation `for: Any` lets us specify a generic fallback.
- With `Eq` we used `id` for identity, but for `Ord` we'll base comparisons on what matters in each case: `name` for rides and patrons, and `time` for FastPasses.

```elixir
# lib/fun_park/ride.ex
defimpl FunPark.Ord, for: FunPark.Ride do
  alias FunPark.Ord
  alias FunPark.Ride

  def lt?(%Ride{name: v1}, %Ride{name: v2}), do: Ord.lt?(v1, v2)
  def le?(%Ride{name: v1}, %Ride{name: v2}), do: Ord.le?(v1, v2)
  def gt?(%Ride{name: v1}, %Ride{name: v2}), do: Ord.gt?(v1, v2)
  def ge?(%Ride{name: v1}, %Ride{name: v2}), do: Ord.ge?(v1, v2)
end

# lib/fun_park/fast_pass.ex
# FastPass is special because it defines `Ord` by a `DateTime`, which is
# just another struct which needs its own implementation of the protocol.
# `DateTime.compare/2` is already a library function that understand time.
# We just have to deal with its return values of `:gt/:lt/:eq`.
defimpl FunPark.Ord, for: FunPark.FastPass do
  alias FunPark.Ord
  alias FunPark.FastPass

  def lt?(%FastPass{time: v1}, %FastPass{time: v2}), do: Ord.lt?(v1, v2)
  def le?(%FastPass{time: v1}, %FastPass{time: v2}), do: Ord.le?(v1, v2)
  def gt?(%FastPass{time: v1}, %FastPass{time: v2}), do: Ord.gt?(v1, v2)
  def ge?(%FastPass{time: v1}, %FastPass{time: v2}), do: Ord.ge?(v1, v2)
end

defimpl FunPark.Ord, for: DateTime do
  def lt?(a, b), do: DateTime.compare(a, b) == :lt
  def le?(a, b), do: match?(x when x in [:lt, :eq], DateTime.compare(a, b))
  def gt?(a, b), do: DateTime.compare(a, b) == :gt
  def ge?(a, b), do: match?(x when x in [:gt, :eq], DateTime.compare(a, b))
end
```

- Note that the match clause would be easier to express without `match` as: `DateTime.compare(a, b) in [:lt, :eq]`, but this displays that you can match on x (which matches anything) and then in a when clause make sure that it is in the list `[:lt, :eq]`.
- Again we see *projection*, mapping a complex structure to a simpler one before applying logic.
- Just so you know, Elixir's default comparison operators compare structs and maps using their size (which is the same for our structs) then by their keys in alphabetical order, so `ride1 < ride2` depends on `id` values since `id` is the first key alphabetically. Assuming

`:calendar`, `:day`, and `:hour` are all the same, `datetime1 < datetime2` compares `:microsecond` values since it comes alphabetically before `:minute`.

- This is easy to use (and a good reminder we wrote a `change` function for our structs):

```
iex> apple_cart = FunPark.Ride.make("Apple Cart")
iex> banana_slip = FunPark.Ride.make("Banana Slip")
iex> datetime_1 = DateTime.new!(~D[2025-06-01], ~T[13:10:00.000005])
iex> datetime_2 = DateTime.new!(~D[2025-06-01], ~T[13:40:00.000005])
iex> fast_pass_1 = FunPark.FastPass.make(banana_slip, datetime_1)
iex> fast_pass_2 = FunPark.FastPass.make(apple_cart, datetime_2)

iex> FunPark.Ord.lt?(fast_pass_1, fast_pass_2) # true

iex> time_3 = DateTime.new!(~D[2025-06-01], ~T[15:00:00.000012])
iex> fast_pass_1 = FunPark.FastPass.change(fast_pass_1, %{time: time_3})

iex> FunPark.Ord.gt?(fast_pass_1, fast_pass_2) # true
```

- We have another special case in Patrons. The default sort is by name, which we saw how to implement. But we need another non-default implementation by contravariant functor so we can sort by ticket tier instead.

```elixir
# lib/fun_park/ord/utils.ex
def contramap(f, ord \\ Ord) do
  ord = to_ord_map(ord)

  %{
    lt?: fn a, b -> ord.lt?.(f.(a), f.(b)) end,
    le?: fn a, b -> ord.le?.(f.(a), f.(b)) end,
    gt?: fn a, b -> ord.gt?.(f.(a), f.(b)) end,
    ge?: fn a, b -> ord.ge?.(f.(a), f.(b)) end,
  }
end

def to_ord_map(%{lt?: lt_f, le?: le_f, gt?: gt_f, ge?: ge_f} = ord_map)
    when is_function(lt_f, 2) and
          is_function(le_f, 2) and
          is_function(gt_f, 2) and
          is_function(ge_f, 2),
  do: ord_map

def to_ord_map(module) when is_atom(module) do
  %{
    lt?: &module.lt?/2,
    le?: &module.le?/2,
    gt?: &module.gt?/2,
    ge?: &module.ge?/2
  }
end

# lib/fun_park/patron.ex
defp tier_priority(:vip), do: 3
defp tier_priority(:premium), do: 2
defp tier_priority(:basic), do: 1
defp tier_priority(_), do: 0

defp get_ticket_tier_priority(%__MODULE__{ticket_tier: ticket_tier}),
  do: tier_priority(ticket_tier)

defp ord_by_ticket_tier do
  Ord.Utils.contramap(&get_ticket_tier_priority/1)
end
```

- Here we are using normal integer comparison by projecting each `ticket-tier` to a numeric value before doing the comparison.

- The contramap operation doesn't change the values it is working with, it just temporarily projects them to a number, returns a comparator that sorts items by that, and leaves the original structure untouched. You can think of the number as paired with the original map structure.

```
iex> alice = FunPark.Patron.make("Alice", 15, 50, ticket_tier: :premium)
iex> beth = FunPark.Patron.make("Beth", 16, 53)

iex> ticket_ord = FunPark.Patron.ord_by_ticket_tier()
iex> ticket_ord.gt?.(alice, beth) # true
# Now beth upgrades to :vip
iex> beth = FunPark.Patron.change(beth, %{ticket_tier: :vip})
iex> ticket_ord.gt?.(beth, alice) # true
```

- It's easy to define an ordering for Patrons by reward points or an ordering of Rides by current wait time.

```
# lib/fun_park/patron.ex
def get_reward_points(%__MODULE__{reward_points: reward_points}),
  do: reward_points

def ord_by_reward_points do
  Ord.Utils.contramap(&get_reward_points/1)
end

# lib/fun_park/ride.ex
def get_wait_time(%__MODULE__{wait_time: wait_time}), do: wait_time

def ord_by_wait_time do
  Ord.Utils.contramap(&get_wait_time/1)
end
```

- We're going to make our `Ord` easy to use by writing a general purpose `compare/2` function. In many languages this would return -1, 0, 1, but as we've seen with `DateTime`, in Elixir it is standard to return `:lt, :eq, :gt`.

```elixir
# lib/fun_park/ord/utils.ex
def compare(a, b, ord \\ Ord) do
  ord = to_ord_map(ord)

  cond do
    ord.lt?.(a, b) -> :lt
    ord.gt?.(a, b) -> :gt
    true -> :eq
  end
end

def comparator(ord_module) do
  fn a, b -> compare(a, b, ord_module) != :gt end
end

# lib/fun_park/list.ex
def sort(list, ord \\ FunPark.Ord) when is_list(list) do
  Enum.sort(list, Ord.Utils.comparator(ord))
end
```

- Above we see when sorting `not :gt` is preferable to `:lt` to establish a stable sort, where equal elements retain their relative positions.
- Our `FunPark.List.sort` works equally well on lists of primitives as it does on domain-specific types.
- A *strict sort* returns a sorted list of unique values. We can easily provide this by composing our existing `uniq/2` and `sort/2` functions. One problem thought is we defined identity by `id` and a `strict sort` requires `Eq` and `Ord` to be aligned. So we must derive `Eq` from `Ord`.

```elixir
# lib/fun_park/ord/utils.ex
def to_eq(ord \\ Ord) do
  %{
    eq?: fn a, b -> compare(a, b, ord) == :eq end,
    not_eq?: fn a, b -> compare(a, b, ord) != :eq end
  }
end

# lib/fun_park/list.ex
def strict_sort(list, ord \\ FunPark.Ord) when is_list(list) do
  list
  |> uniq(Ord.Utils.to_eq(ord))
  |> sort(ord)
end
```

- So if we wanted a sorted list of rides by wait time with only one ride per wait time, we could do the following:

```elixir
iex> tea_cup = FunPark.Ride.make("Tea Cup", wait_time: 40)
iex> haunted_mansion = FunPark.Ride.make("Haunted Mansion", wait_time: 20)
iex> river_ride = FunPark.Ride.make("River Ride", wait_time: 40)
iex> rides = [tea_cup, haunted_mansion, river_ride]

iex> ord_wait_time = FunPark.Ride.ord_by_wait_time()
iex> FunPark.List.sort(rides, ord_wait_time)
# [
# %FunPark.Ride{ name: "Haunted Mansion", wait_time: 20, ... },
# %FunPark.Ride{ name: "Tea Cup", wait_time: 40, ... },
# %FunPark.Ride{ name: "River Ride", wait_time: 40, ... },
# ]
iex> FunPark.List.strict_sort(rides, ord_wait_time)
# [
# %FunPark.Ride{ name: "Haunted Mansion", wait_time: 20, ... },
# %FunPark.Ride{ name: "Tea Cup", wait_time: 40, ... },
# ]
```

- Sorting then reversing is inefficient compared to being able to reverse an: `Ord`.

```
# lib/fun_park/ord/utils.ex
def reverse(ord \\ Ord) do
  ord = to_ord_map(ord)

  %{
    lt?: ord.gt?,
    le?: ord.ge?,
    gt?: ord.lt?,
    ge?: ord.le?,
  }
end

iex> ticket_ord = FunPark.Patron.ord_by_ticket_tier()
iex> reverse_ticket_ord = FunPark.Ord.Utils.reverse(ticket_ord)
```

- Since `Eq` and `Ord` are mostly boilerplate, we can automate our typical use case with a macro rather than implementing `Ord` for every struct.

```elixir
# lib/fun_park/macros.ex
defmacro ord_for(for_struct, field) do
  quote do
    alias FunPark.Ord

    defimpl FunPark.Ord, for: unquote(for_struct) do
      def lt?(
        %unquote(for_struct){unquote(field) => v1},
        %unquote(for_struct){unquote(field) => v2},
      ),
      do: Ord.lt?(v1, v2)

      def le?(
        %unquote(for_struct){unquote(field) => v1},
        %unquote(for_struct){unquote(field) => v2},
      ),
      do: Ord.le?(v1, v2)

      def gt?(
        %unquote(for_struct){unquote(field) => v1},
        %unquote(for_struct){unquote(field) => v2},
      ),
      do: Ord.gt?(v1, v2)

      def ge?(
        %unquote(for_struct){unquote(field) => v1},
        %unquote(for_struct){unquote(field) => v2},
      ),
      do: Ord.ge?(v1, v2)
    end
  end
end

# Then to use it, we just do following:
# lib/fun_park/patron.ex
ord_for(FunPark.Patron, :name)

# Likewise, we could have an `Eq` macro.
# lib/fun_park/macros.ex
defmacro eq_for(for_struct, field) do
  quote do
    alias FunPark.Eq

    defimpl FunPark.Eq, for: unquote(for_struct) do
      def eq?(
```

```
        %unquote(for_struct){unquote(field) => v1},
        %unquote(for_struct){unquote(field) => v2},
      ),
      do: Eq.eq?(v1, v2)

      def not_eq?(
        %unquote(for_struct){unquote(field) => v1},
        %unquote(for_struct){unquote(field) => v2},
      ),
      do: Eq.not_eq?(v1, v2)
    end
  end
end

# We can implement min/2 and max/2 easy enough in our utils.
# lib/fun_park/ord/utils.ex
def min(a, b, ord \\ Ord) do
  case compare(a, b, ord) do
    :gt -> b
    _ -> a
  end
end

def max(a, b, ord \\ Ord) do
  case compare(a, b, ord) do
    :lt -> b
    _ -> a
  end
end
```

Chapter 4 - Combine with Monoids

- Many simple comparison problems are actually combination problems. Who is first out of someone with a FastPass, a VIP, and a rider with accessibility needs. With *monoids* we get a reusable abstraction to explicitly combine.
- A *semigroup* defines how elements combine and satisfies two rules:
  - Associativity: `a + (b + c) = (a + b) + c`. Grouping doesn't change result.
  - Closure: Combining two elements produces another element of the same kind.
- A *monoid* adds a third rule:
  - Identity: There's a neutral element `e` such that `a + e = a`.

- Conceptually, a monoid extends a semigroup, but in Elixir protocols are defined independently.
- Some types have a well-understood way to combine, such as concatenation for Lists. Others have multiple valid ways, such as addition, multiplication, maximum, or minimum in regards to Numbers. To make the combination strategy explicit in Elixir, we use a struct.
- Like many functional programming concepts, the underlying ideas are consistent while names vary across languages and libraries. An identity element might be called **identity**, **start**, or **zero**. A combination could be **append**, **concat**, **combine**, or **add**. We will continue to follow Haskell naming conventions and adopt `Monoid.empty` and `Monoid.append`.

```elixir
# A Monoid must define empty/1 and append/2.
# lib/fun_park/monoid.ex
defprotocol FunPark.Monoid do
  def empty(monoid_struct)
  def append(monoid_struct_a, monoid_struct_b)
  def wrap(monoid_struct, value)
  def unwrap(monoid_struct)
end
```

- Since the protocol dispatches on struct type, `wrap` and `unwrap` provide a way to move raw values in and out of a struct wrapper.
- Let's look at a concrete example in code, the `Sum` Monoid.

```elixir
# lib/fun_park/monoid/sum.ex
defmodule FunPark.Monoid.Sum do
  defstruct value: 0
end

defimpl FunPark.Monoid, for: FunPark.Monoid.Sum do
  alias FunPark.Monoid.Sum

  def empty(_), do: %Sum{}

  def append(%Sum{value: a}, %Sum{value: b}) do
    %Sum{value: a + b}
  end

  def wrap(%Sum{}, value) when is_number(value), do: %Sum{value: value}

  def unwrap(%Sum{value: value}) when is_number(value), do: value
end
```

- While nothing stops us from constructing Sum's directly, we avoid it and use `wrap/2` which lifts a raw value into the monoid context without exposing internal details.

```elixir
iex> sum_1 = FunPark.Monoid.wrap(%FunPark.Monoid.Sum{}, 1)
iex> sum_2 = FunPark.Monoid.wrap(%FunPark.Monoid.Sum{}, 2)
iex> value = FunPark.Monoid.append(sum_1, sum_2)
# %FunPark.Monoid.Sum{value: 3}
iex> FunPark.Monoid.unwrap(value) # 3
```

- A caller shouldn't need to worry about wrapping and unwrapping, so we define a `m_append` helper function in Monoid.Utils.

```elixir
# lib/fun_park/monoid/utils.ex
import FunPark.Monoid, only: [empty: 1, append: 2, wrap: 2, unwrap: 1]

def m_append(monoid, a, b) when is_struct(monoid) do
  append(wrap(monoid, a), wrap(monoid, b)) |> unwrap()
end

iex> FunPark.Monoid.Utils.m_append(%FunPark.Monoid.Sum{}, 1, 2) # 3
```

- For a list, folding requires three components: the structure to reduce, a function to combine its contents, and a base case. A `Monoid` provides `append/2` for combination and `empty/1` for the identity.

```
# lib/fun_park/foldable.ex
defprotocol FunPark.Foldable do
  def fold_l(structure, base, transform_fn)
  def fold_r(structure, base, transform_fn)
end

# lib/fun_park/list.ex
defimpl FunPark.Foldable, for: List do
  def fold_l(list, acc, func), do: :lists.foldl(func, acc, list)
  def fold_r(list, acc, func), do: :lists.foldr(func, acc, list)
end

# lib/fun_park/utils.ex
def m_concat(monoid, values) when is_struct(monoid) and is_list(values) do
  fold_l(values, empty(monoid), fn value, acc ->
    append(acc, wrap(monoid, value))
  end)
  |> unwrap
end
```

- Monoids are typically abstracted from the caller with a function that hides the details, so we could provide a simple `Math` module.

```
# lib/fun_park/math.ex
def sum(a, b) do
  m_append(%Monoid.Sum{}, a, b)
end

def sum(list) when is_list(list) do
  m_concat(%Monoid.Sum{}, list)
end

iex> FunPark.Math.sum(1, 2) # 3
iex> FunPark.Math.sum([1, 2, 3]) # 6
iex> FunPark.Math.sum([3]) # 3
iex> FunPark.Math.sum([]) # 0
```

- `m_concat` uses the list's Foldable implementation to walk each raw element; for every element, the fold function wraps it into a `Sum{}` and immediately `append`s it onto the running accumulator, which itself starts as empty(monoid).
- In this case, the List isn't actually participating in the Monoid at all. It merely holds elements that participate in Sum Monoid and allows us to walk through them combining via their `append` since it implements `Foldable`. `Foldable` is the concept of walkable here being applied to Lists, and `Sum` is the concept of combinable with an identity, with identity being used for the initial `acc` and combinable being `append`. We could use any Monoid here for our combining and identity, to say find the `Max` of a List or the `Product`, and m_concat would require no changes.
- So we've seen combining numbers and combining list elements in a `Sum` Monoid, but what about combining `Eq`. There are two ways to combine `Eq`:
  - **All**: Every equality check must return `true`.
  - **Any**: At least one equality check must return `true`.
- **All** is biased towards `true`. It starts `true` and flips if any condition fails. **Any** is biased towards `false`. It starts from `false` and flips if any condition passes.

```elixir
# lib/fun_park/monoid/eq_all.ex
defmodule FunPark.Monoid.Eq.All do
  defstruct eq?: &FunPark.Monoid.Eq.All.default_eq?/2,
            not_eq?: &FunPark.Monoid.Eq.All.default_not_eq?/2

  def default_eq?(_, _), do: true
  def default_not_eq?(_, _), do: false
end

defimpl FunPark.Monoid, for: FunPark.Monoid.Eq.All do
  alias FunPark.Eq.Utils
  alias FunPark.Monoid.Eq.All

  def empty(_), do: %All{}

  def append(%All{} = eq1, %All{} = eq2) do
    %All{
      eq?: fn a, b -> eq1.eq?.(a, b) && eq2.eq?.(a, b) end, # AND
      not_eq?: fn a, b -> eq1.not_eq?.(a, b) || eq2.not_eq?.(a, b) end # OR
    }
  end

  def wrap(%All{}, eq) do
    eq = Utils.to_eq_map(eq)
    %All{
      eq?: eq.eq?,
      not_eq?: eq.not_eq?
    }
  end

  def unwrap(%All{eq?: eq?, not_eq?: not_eq?}) do
    %{eq?: eq?, not_eq?: not_eq?}
  end
end

# lib/fun_park/eq/utils.ex
def append_all(a, b) do
  m_append(%Monoid.Eq.All{}, a, b)
end

def concat_all(eq_list) when is_list(eq_list) do
  m_concat(%Monoid.Eq.All{}, eq_list)
end
```

- So `Eq.All` isn't even a relationship where we wrap numbers and then compare them. It's a relationship where we wrap up notions of equality and compose them into more complicated notions of equality. In the end it just returns a super-function that applies multiple notions of equality to whatever non-Monoid participant domain values you provide it. And it's `m_concat` is designed to work on a list of Eq maps. The identity element is just an `Eq` map with functions always returning `true` / `false` for `eq?` / `not_eq?`.

- It's at this point that I'll insert a personal observation by the author verbatim. "I have a soft spot for monoids - they were the first time I realized functional programming wasn't just about constructing pipelines to transform values but also about composing *concepts: rules, behaviors, and strategies. They're not just a way to structure code but a way to structure thought."

```
# But how do we use it?
iex> datetime = DateTime.new!(~D[2025-06-01], ~T[13:00:00])
iex> apple = FunPark.Ride.make("Apple Cart")
iex> fast_pass_a = FunPark.FastPass.make(apple, datetime)
iex> fast_pass_b = FunPark.FastPass.make(apple, datetime)

iex> eq_ride = FunPark.FastPass.eq_ride()
iex> eq_time = FunPark.FastPass.eq_time()
iex> eq_both = FunPark.Eq.Utils.concat_all([eq_ride, eq_time])

# Both have unique `id` values, so they are different by the default `Eq`.
iex> FunPark.Eq.Utils.eq?(fast_pass_a, fast_pass_b) # false
iex> FunPark.Eq.Utils.eq?(fast_pass_a, fast_pass_b, eq_both)

iex> datetime_2 = DateTime.new!(~D[2025-06-01], ~T[14:00:00])
iex> fast_pass_a = FunPark.FastPass.change(fast_pass_a, %{time: datetime_2})
iex> FunPark.Eq.Utils.eq?(fast_pass_a, fast_pass_b, eq_both) # false
```

- If this is a common comparison, it belongs in the FastPass bounded context.

```
# lib/fun_park/fast_pass.ex
def eq_ride_and_time do
  Eq.Utils.concat_all([eq_ride(), eq_time()])
end
```

- But our domain expert corrected us, we really wanted to still compare two FastPasses with the same id as equal but also consider them equal if they are for the same ride and time. So we need `id || ride && time`.

```elixir
# lib/fun_park/monoid/eq_any.ex
defmodule FunPark.Monoid.Eq.Any do
  defstruct eq?: &FunPark.Monoid.Eq.Any.default_eq?/2,
            not_eq?: &FunPark.Monoid.Eq.Any.default_not_eq?/2

  def default_eq?(_, _), do: false
  def default_not_eq?(_, _), do: true
end

defimpl FunPark.Monoid, for: FunPark.Monoid.Eq.Any do
  alias FunPark.Eq.Utils
  alias FunPark.Monoid.Eq.Any

  def empty(_), do: %Any{}

  def append(%Any{} = eq1, %Any{} = eq2) do
    %Any{
      eq?: fn a, b -> eq1.eq?.(a, b) || eq2.eq?.(a, b) end, # OR
      not_eq?: fn a, b -> eq1.not_eq?.(a, b) && eq2.not_eq?.(a, b) end # AND
    }
  end

  def wrap(%Any{}, eq) do
    eq = Utils.to_eq_map(eq)

    %Any{
      eq?: eq.eq?,
      not_eq?: eq.not_eq?
    }
  end

  def unwrap(%Any{eq?: eq?, not_eq?: not_eq?}) do
    %{eq?: eq?, not_eq?: not_eq?}
  end
end

# In Eq.Utils we abstract away the Monoid details.
# lib/fun_park/eq/utils.ex
def append_any(a, b) do
  m_append(%Monoid.Eq.Any{}, a, b)
end

def concat_any(eq_list) do
  m_concat(%Monoid.Eq.Any{}, eq_list)
end
```

```elixir
# lib/fun_park/fast_pass.ex
def duplicate_pass do
  Eq.Utils.concat_any([Eq, eq_ride_and_time()])
end
```

- Our final duplicate_pass definition is declarative. It describes what makes two passes duplicates and leaves the monoids to handle how the comparison is performed.

```elixir
iex> datetime = DateTime.new!(~D[2025-06-01], ~T[13:00:00])
iex> tea_cup = FunPark.Ride.make("Tea Cup")
iex> pass_a = FunPark.FastPass.make(tea_cup, datetime)
iex> pass_b = FunPark.FastPass.make(tea_cup, datetime)
iex> FunPark.Eq.Utils.eq?(pass_a, pass_b) # false; id's are different
iex> dup_pass_check = FunPark.FastPass.duplicate_pass()
iex> FunPark.Eq.Utils.eq?(pass_a, pass_b, dup_pass_check) # true; ride/time
iex> mansion = FunPark.Ride.make("Haunted Mansion")
iex> pass_a_changed = FunPark.FastPass.change(pass_a, %{ride: mansion})
iex> FunPark.Eq.Utils.eq?(pass_a, pass_a_changed, dup_pass_check) # true; id
iex> FunPark.Eq.Utils.eq?(pass_b, pass_a_changed, dup_pass_check) # false
```

- Combining `Ord` instances is a hard problem that Monoid makes easy for us. The identity for `Ord` is `false` for everything, and the bias for `Ord` is *incomparable*: elements are assumed unordered unless a rule defines their relationship.

```elixir
# lib/fun_park/monoid/ord.ex
defmodule FunPark.Monoid.Ord do
  defstruct lt?: &FunPark.Monoid.Ord.default?/2,
            le?: &FunPark.Monoid.Ord.default?/2,
            gt?: &FunPark.Monoid.Ord.default?/2,
            ge?: &FunPark.Monoid.Ord.default?/2

  def default?(_, _), do: false
end

defimpl FunPark.Monoid, for: FunPark.Monoid.Ord do
  alias FunPark.Monoid.Ord
  alias FunPark.Ord.Utils

  def empty(_) do
    %Ord{}
  end

  # To combine two orderings, you just test the first
  # and fall through to the second for equality.
  def append(%Ord{} = ord1, %Ord{} = ord2) do
    %Ord{
      lt?: fn a, b ->
        cond do
          ord1.lt?.(a, b) -> true
          ord1.gt?.(a, b) -> false
          true -> ord2.lt?.(a, b)
        end
      end,
      le?: fn a, b ->
        cond do
          ord1.lt?.(a, b) -> true
          ord1.gt?.(a, b) -> false
          true -> ord2.le?.(a, b)
        end
      end,
      gt?: fn a, b ->
        cond do
          ord1.gt?.(a, b) -> true
          ord1.lt?.(a, b) -> false
          true -> ord2.gt?.(a, b)
        end
      end,
      ge?: fn a, b ->
        cond do
```

```
          ord1.gt?.(a, b) -> true
          ord1.lt?.(a, b) -> false
          true -> ord2.ge?.(a, b)
        end
      end
    }
  end

  def wrap(%Ord{}, ord) do
    ord = Utils.to_ord_map(ord)

    %Ord{
      lt?: ord.lt?,
      le?: ord.le?,
      gt?: ord.gt?,
      ge?: ord.ge?
    }
  end

  def unwrap(%Ord{lt?: lt?, le?: le?, gt?: gt?, ge?: ge?}) do
    %{
      lt?: lt?,
      le?: le?,
      gt?: gt?,
      ge?: ge?
    }
  end
end

# lib/fun_park/ord/utils.ex
def append(a, b) do
  m_append(%FunPark.Monoid.Ord{}, a, b)
end

def concat(ord_list) when is_list(ord_list) do
  m_concat(%FunPark.Monoid.Ord{}, ord_list)
end

iex> ord_ticket = FunPark.Patron.ord_by_ticket_tier()
iex> ord_reward_points = FunPark.Patron.ord_by_reward_points()
iex> ord_priority = FunPark.Ord.Utils.concat(
    [ord_ticket, ord_reward_points, FunPark.Ord]
)
iex> FunPark.List.sort([charles, beth, alice], ord_priority)
# returns result of ordering by ticket_tier then reward_points then name.
```

```elixir
# lib/fun_park/patron.ex
def ord_by_priority do
  Ord.Utils.concat([
    ord_by_ticket_tier(),
    ord_by_reward_points(),
    Ord
  ])
end
```

- The `Monoid.Max` encapsulates both the value and the ordering logic for finding the maximum and is defined in `lib/fun_park/monoid/max.ex`. It's not much code so I'll list it here too. Worth pointing out is `min_value` has to be passed to `empty` by the user since minium for a domain isn't something the code can just guess and when you `append` together two `Monoid.Max` their `Ord` must be equal to be joined so the `Ord` of the second is ignored. It's meaningless to try to determine `Max` with two separate `Ord` orderings.

```elixir
# lib/fun_park/monoid/max.ex
defmodule FunPark.Monoid.Max do
  defstruct value: nil, ord: FunPark.Ord
end

def empty(%Max{value: min_value, ord: ord}) do
  %Max{value: min_value, ord: ord}
end

# lib/fun_park/ord/utils.ex
def max(a, b, ord \\ Ord) do
  case compare(a, b, ord) do
    :lt -> b
    _ -> a
  end
end

# lib/fun_park/monoid/max.ex
def append(%Max{value: a, ord: ord}, %Max{value: b}) do
  %Max{value: Utils.max(a, b, ord), ord: ord}
end

def wrap(%Max{ord: ord}, value) do
  %Max{value: value, ord: Utils.to_ord_map(ord)}
end

def unwrap(%Max{value: value}), do: value

# lib/fun_park/math.ex
# We will have to assume we're working with numbers. This is Math.
def max(a, b) do
  m_append(%Monoid.Max{value: Float.min_finite()}, a, b)
end

def max(list) when is_list(list) do
  m_concat(%Monoid.Max{value: Float.min_finite()}, list)
end

# We can use this in our domain for a Priority queue for Patrons.
# lib/fun_park/patron.ex
def priority_empty do
  %__MODULE__{reward_points: Float.min_finite(), ticket_tier: nil}
end

defp max_priority_monoid do
```

```
  %Monoid.Max{
    value: priority_empty(),
    ord: ord_by_priority() # first ticket_tier, then reward_points
  }
end

# We want to abstract the monoid
def highest_priority(patrons) when is_list(patrons) do
  m_concat(max_priority_monoid(), patrons)
end

iex> alice = FunPark.Patron.make("Alice", 15, 150)
iex> beth = FunPark.Patron.make("Beth", 15, 150, reward_points: 100)
iex> FunPark.Patron.highest_priority([beth, alice]) # beth by reward_points
iex> alice = FunPark.Patron.change(alice, %{ticket_tier: :vip})
iex> FunPark.Patron.highest_priority([beth, alice]) # alice by ticket_tier
```

- Unlike Elixir's `max/2`, our `Monoid.Max` works with lists of one element. We currently return the sentinel value for an empty list and require the caller to know this is a placeholder. We will explore `Maybe` later.
- `Product` and `Min` Monoids are straightforward after `Sum` and `Max`.
- Software complexity can be measured as *cyclomatic complexity*, the number of independent paths through a function. A core of 1-4 is considered easy to maintain, 5-10 suggests moderate complexity where changes require careful consideration, and above 10 signals high complexity which is difficult to extend and maintain. An imperative implementation of `ord_by_priority` scores a cyclomatic complexity of 9 easily, and where do you start when a new business rule is introduced such as patrons with 50 or more reward points receive the same priority of those with a ticket upgrade. And then the rule gets modified, reward points trump `:basic` to `:premium` but not `:vip`. Every developer that makes a change needs to navigate the complexity anew. Declarative code like we've been writing focuses on what should happen instead of how to do it. We could've written `ord_by_priority` as:

```
def ord_by_priority do
  ord_by_ticket_tier()
  |> append(ord_by_reward_points())
  |> append(Ord)
end
```

- The above has cyclomatic complexity of 1, the lowest possible. If the domain rules change it all changes in one place via easily composable functions. Because the logic is declarative and consistent, it's easier for teams to understand, adapt, and extend together.
- `Eq, Ord, Monoid` aren't just abstractions; they're mental models. The more we use them, the more they shape how we see problems: as equality, ordering, or combination. And once we see the shape of a problem we can reason about it more clearly.

Chapter 5 - Define Logic with Predicates

- This chapter adds several new mental models: treating predicates as composable structures, applying logic across domain boundaries, and composing functions whose shapes don't align.
- Elixir also has some idioms that complicate functional solutions: no built-in currying, a pipe operator that targets the first argument, and different syntax for named and anonymous functions.
- A *predicate* is a statement that can be true or false within a context. Because they follow Boolean algebra, they compose:
    - Conjunction (a && b): True if both are true.
    - Disjunction (a || b): True if at least one is true.
    - Negation (!a): Inverts the result.
- Complex policies, dynamic filters, validation pipelines - they all follow the same idea: combining small checks into larger ones.

```elixir
# lib/fun_park/ride.ex
# We need to know if a Ride is online and if it has a long wait.
def online?(%__MODULE__{online: online}), do: online
def long_wait?(%__MODULE__{wait_time: wait_time}), do: wait_time > 30

# But we also need short wait, defined as the inverse of `long_wait?`
# for consistency.

# lib/fun_park/predicate.ex
# inverse is a useful general concept.
def p_not(pred) when is_function(pred) do
  fn value -> not pred.(value) end
end

# lib/fun_park/ride.ex
def short_wait?, do: p_not(&long_wait?/1)

# Now a new rule. A ride is suggested if it's online and has a short wait.
```

- Predicate logic has well-defined rules for combination.
  - **and** - Combines two predicates, returning `true` if both are `true`.
  - **or** - Combines two predicates, returning `true` if one is `true`.
  - **all** - Returns `true` if every predicate in a set returns `true`.
  - **any** - Returns `true` if at least one predicate in a set returns `true`.
  - **none** - The inverse of **any**, returning `true` if none return `true`.
- We have the perfect tool for combination - monoids! Predicate composition only needs `Predicate.All` and `Predicate.Any`. `All` is biased towards `true` and `Any` is biased towards `false`.
- `Predicate.All` combines predicates with conjunction using an identity of `() -> true`.

```elixir
# lib/fun_park/monoid/pred_all.ex
defmodule FunPark.Monoid.Predicate.All do
  defstruct value: &FunPark.Monoid.Predicate.All.default_pred?/1

  def default_pred?(_), do: true
end

defimpl FunPark.Monoid, for: FunPark.Monoid.Predicate.All do
  alias FunPark.Monoid.Predicate.All

  def empty(_), do: %All{}

  def append(%All{} = p1, %All{} = p2) do
    %All{
      value: fn value -> p1.value.(value) and p2.value.(value) end
    }
  end

  def wrap(%All{}, value) when is_function(value, 1) do
    %All{value: value}
  end

  def unwrap(%All{value: value}), do: value
end
```

- `Predicate.Any` combines predicates with disjunction using an identity of `() -> false`.

```elixir
# lib/fun_park/monoid/pred_any.ex
defmodule FunPark.Monoid.Predicate.Any do
  defstruct value: &FunPark.Monoid.Predicate.Any.default_pred?/1

  def default_pred?(_), do: false
end

defimpl FunPark.Monoid, for: FunPark.Monoid.Predicate.Any do
  alias FunPark.Monoid.Predicate.Any

  def empty(_), do: %Any{}

  def append(%Any{} = p1, %Any{} = p2) do
    %Any{
      value: fn value -> p1.value.(value) or p2.value.(value) end
    }
  end

  def wrap(%Any{}, value) when is_function(value, 1) do
    %Any{value: value}
  end

  def unwrap(%Any{value: value}), do: value
end
```

- One of the best parts about monoids is they are closed under their operation, so with predicates, every composition returns a predicate.

```elixir
# Now we need to hide the monoids
# lib/fun_park/predicate.ex
defmodule FunPark.Predicate do
  import FunPark.Monoid.Utils, only: [m_append: 3, m_concat: 2]
  alias FunPark.Monoid.Predicate.{All, Any}

  def p_and(pred1, pred2) when is_function(pred1) and is_function(pred2) do
    m_append(%All{}, pred1, pred2)
  end

  def p_or(pred1, pred2) when is_function(pred1) and is_function(pred2) do
    m_append(%Any{}, pred1, pred2)
  end

  def p_not(pred) when is_function(pred) do
    fn value -> not pred.(value) end
  end

  def p_all(p_list) when is_list(p_list) do
    m_concat(%All{}, p_list)
  end

  def p_any(p_list) when is_list(p_list) do
    m_concat(%Any{}, p_list)
  end

  def p_none(p_list) when is_list(p_list) do
    p_not(p_any(p_list))
  end
end
```

- Now we have the machinery we need to combine `online?/1` and `long_wait?/1`:

```elixir
# lib/fun_park/ride.ex
def suggested?(%__MODULE__{} = ride),
  do: p_all([&online?/1, p_not(&long_wait?/1)]).(ride)
```

- We are using a monoid to combine ideas. A predicate isn't a Boolean; it's a function that returns a Boolean.
- But now we need to span bounded contexts! We're suggesting rides to Patrons that can't take the ride. When bounded contexts interact their relationship is defined by *context*

mapping. In FunPark, the Ride context sets the eligibility rules such as height and age requirements and the Patron context supplies height and age attributes. This forms a *conformist relationship* where Patron conforms to the rules set by Ride but has no influence over them. Since Ride defines the rules, the logic for determining eligibility belongs in the Ride bounded context.

```
# lib/fun_park/ride.ex
def tall_enough?(%Patron{} = patron, %__MODULE__{min_height: min_height}),
  do: Patron.get_height(patron) >= min_height

def old_enough?(%Patron{} = patron, %__MODULE__{min_age: min_age}),
  do: Patron.get_age(patron) >= min_age

def eligible?(%Patron{} = patron, %__MODULE__{} = ride),
  do: p_all([&tall_enough?/2, &old_enough?/2]).(patron, ride)
```

- Notice above we don't destructure Patron to access `age` or `height`. That's an implementation detail we should not rely on. We use the Patron's accessors instead. This keeps the contexts loosely coupled.

```
iex> roller_mtn = FunPark.Ride.make(
  "Roller Mountain", min_height: 120, min_age: 12
)
iex> alice = FunPark.Patron.make("Alice", 13, 119)
iex> alice |> FunPark.Ride.old_enough?(roller_mtn) # true
iex> alice |> FunPark.Ride.tall_enough?(roller_mtn) # false
iex> alice |> FunPark.Ride.eligible?(roller_mtn) # false
# Now alice grows over the summer.
iex> alice = FunPark.Patron.change(alice, %{height: 121})
iex> alice |> FunPark.Ride.eligible?(roller_mtn) # true
```

- Elixir is unusual among functional languages because it allows direct composition of functions with multiple arguments. In most languages, function composition is limited to *unary* functions.

- But now we need to compose a two argument `eligible?/2` function with a one argument `suggested?/1` function. The solution is currying.

```elixir
def curry(fun) when is_function(fun) do
  arity = :erlang.fun_info(fun, :arity) |> elem(1)
  curry(fun, arity, [])
end

defp curry(fun, 1, args),
  do: fn last_arg -> apply(fun, args ++ [last_arg]) end

defp curry(fun, arity, args) when arity > 1 do
  fn next_arg -> curry(fun, arity - 1, args ++ [next_arg]) end
end
```

- The way this works is simple. If the function is unary, wrap it as-is. Otherwise, apply arguments one at a time, recursively returning a new function until all arguments are provided. So now we can combine the two functions with different arities. But we need to curry all binary predicates before passing them to `p_all` to prevent different arity functions from breaking the chain.

```elixir
# lib/fun_park/ride.ex
def suggested?(%__MODULE__{} = ride),
  do: p_all([&online?/1, p_not(&long_wait?/1)]).(ride)

def suggested?(%Patron{} = patron, %__MODULE__{} = ride),
  do:
    p_all([
      &suggested?/1,
      curry(&eligible?/2).(patron)
    ]).(ride)

def eligible?(%Patron{} = patron, %__MODULE__{} = ride),
  do:
    p_all([
      curry(&tall_enough?/2).(patron),
      curry(&old_enough?/2).(patron),
    ]).(ride)
```

- Unlike `Eq` or `Ord`, Elixir's `Enum` module integrates seamlessly with predicates without extra logic or wrapping. This includes:
  - `Enum.all?/2` - returns `true` if all elements satisfy the predicate.
  - `Enum.any?/2` - returns `true` if any element satisfies the predicate.

- `Enum.count/2` - counts elements that satisfy the predicate.
- `Enum.drop_while/2` - drops elements from the beginning while pred `true`.
- `Enum.filter/2` - returns a list of elements that satisfy the predicate.
- `Enum.find/2` - returns the first element that satisfies the predicate.
- `Enum.find_index/2` - returns the index of the first `true` element.
- `Enum.reject/2` - returns list of elements where the predicate is `false`.
- `Enum.take_while/2` - takes elements from the beginning while pred `true`.
- `Enum.split_while/2` - splits a list at the first `false` element.

```
# lib/fun_park/ride.ex
# So we can use filter for our list of suggested rides.
def suggested_rides(%Patron{} = patron, rides) when is_list(rides) do
  Enum.filter(rides, &suggested?(patron, &1))
end
```

- So with the FastPass there are business rules that govern who gets fast lane access and under what conditions. These rules span three bounded contexts: Patron, FastPass, and Ride. The Patron manages the collection of passes, the FastPass defines what makes one valid, and the Ride determines access to the fast lane. We'll later lift these same rules into monads for richer forms of validation.
- So the Patron needs to manage the collection of passes:

```
# lib/fun_park/patron.ex
def add_fast_pass(%__MODULE__{} = patron, fast_pass) do
  fast_passes = List.union([fast_pass], get_fast_passes(patron))

  change(patron, %{fast_passes: fast_passes})
end

def remove_fast_pass(%__MODULE__{} = patron, fast_pass) do
  fast_passes = List.difference(get_fast_passes(patron), [fast_pass])

  change(patron, %{fast_passes: fast_passes})
end
```

- And the fastPass defines the rules for whether a pass is valid:

```elixir
# lib/fun_park/fast_pass.ex
def get_ride(%__MODULE__{ride: ride}), do: ride

def valid?(%__MODULE__{} = fast_pass, %Ride{} = ride) do
  Eq.Utils.eq?(get_ride(fast_pass), ride)
end
```

- And the Ride context determines whether a patron can enter the fast lane:

```elixir
# lib/fun_park/ride.ex
def fast_pass?(%Patron{} = patron, %__MODULE__{} = ride) do
  patron
  |> Patron.get_fast_passes()
  |> Enum.any?(&FastPass.valid?(&1, ride))
end
```

- So now we're letting people into the fast lane for rides they aren't eligible for. We need to combine `fast_pass?/2` with `eligible?/2`.

```elixir
def fast_pass_lane?(%Patron{} = patron, %__MODULE__{} = ride) do
  has_fast_pass = curry(&fast_pass?/2).(patron)
  is_eligible = curry(&eligible?/2).(patron)

  p_all([has_fast_pass, is_eligible]).(ride)
end
```

- But we forgot VIP patrons always have access to the fast lane if they are eligible for the Ride.

```elixir
# lib/fun_park/patron.ex
def vip?(%__MODULE__{ticket_tier: :vip}), do: true
def vip?(%__MODULE__{}), do: false
```

- But now we have a problem. `has_fast_pass/2` and `is_eligible/2` take a Patron as their first argument and a Ride as their second. But `vip?/1` takes a Patron as its only argument, not a Ride, so doesn't fit in the pipeline. We can use `curry_r/1` to curry from right to left so we have three functions that accept a Patron.

```elixir
# lib/fun_park/utils.ex
def curry_r(fun) when is_function(fun) do
  arity = :erlang.fun_info(fun, :arity) |> elem(1)
  curry_r(fun, arity, [])
end

defp curry_r(fun, 1, args),
  do: fn last_arg -> apply(fun, [last_arg | args]) end

defp curry_r(fun, arity, args) when arity > 1 do
  fn next_arg -> curry_r(fun, arity - 1, [next_arg | args]) end
end
```

- Functional languages often place the data structure as the last argument in a function. Elixir flips this pattern - placing the data first. The choice is reflected in its pipe operator. Therefore `curry_r/1` which applies arguments from right to left is often a better fit for Elixir's conventions.

```elixir
# lib/fun_park/ride.ex
def fast_pass_lane?(%Patron{} = patron, %__MODULE__{} = ride) do
  has_fast_pass = curry_r(&fast_pass?/2).(ride)
  is_eligible = curry_r(&eligible?/2).(ride)
  is_vip = &Patron.vip?/1

  p_all([is_eligible, p_any([is_vip, has_fast_pass])]).(patron)
end
```

```elixir
iex> alice = FunPark.Patron.make("Alice", 13, 150)
iex> beth = FunPark.Patron.make("Beth", 15, 110)

iex> haunted_mansion = FunPark.Ride.make("Haunted Mansion", min_age: 14)
iex> datetime = DateTime.new!(~D[2025-06-01], ~T[13:00:00])
iex> fast_pass = FunPark.FastPass.make(haunted_mansion, datetime)
iex> alice = FunPark.Patron.add_fast_pass(alice, fast_pass)
# alice has a fast pass, but is too young; beth doesn't have a fast pass
iex> alice |> FunPark.Ride.fast_pass_lane?(haunted_mansion) # false
iex> beth |> FunPark.Ride.fast_pass_lane?(haunted_mansion) # false
# but beth upgraded to VIP!
iex> beth = FunPark.Patron.change(beth, %{ticket_tier: :vip})
iex> beth |> FunPark.Ride.fast_pass_lane?(haunted_mansion) # true
```

- By modeling FastPass as its own bounded context and treating Patron as a conformist, we keep responsibilities clear, allowing each part of the system to evolve independently as our understanding of the domain evolves.
- Predicates are also Foldable, but they aren't ordered, so `fold_l/3` and `fold_r/3` behave the same.

```
# lib/fun_park/predicate.ex
defimpl FunPark.Foldable, for: Function do
  def fold_l(predicate, true_func, false_func) do
    case predicate.() do
      true -> true_func.()
      false -> false_func.()
    end
  end

  def fold_r(predicate, true_func, false_func) do
    fold_l(predicate, true_func, false_func)
  end
end
```

- Foldable behaves differently for fixed-shape types — Maybe (Just/Nothing), a predicate (true/false) — than for types like List that have no fixed shape count. Fixed-shape fold takes the item plus one function per possible shape, and dispatches to whichever function matches.
- We implement Foldable `for: Function`, not `for: Predicate`, because there's no `Predicate` type to dispatch on — a predicate here is just a zero-arity `Function` that happens to return a boolean by convention. Elixir's protocol dispatch works on the actual runtime type, and it can't distinguish "a Function" from "a Function that returns Boolean," so `Function` is the only thing actually available to implement against. Note this zero-arity predicate is a thunk — we have to evaluate it (call it with no args) to fold over it, unlike the one-argument predicates we built in Chapter 5.

```
iex> tea_cup = FunPark.Ride.make("Tea Cup", online: true, wait_time: 100)
iex> FunPark.Ride.suggested?(tea_cup) # false; wait too long
iex> yes_or_no = fn val, pred ->
  FunPark.Foldable.fold_l(fn ->
    pred.(val) end, fn -> "Yes" end, fn -> "No" end) end
iex> yes_or_no.(tea_cup, &FunPark.Ride.suggested?/1) # No
```

- Now beyond `suggested?/2` we want to implement a recommendation function that both checks if the ride is `online?/1` and has a short `wait_time?/1` and if the Patron is `eligible?/2` but also takes into consideration the Patron's `:likes` and `:dislikes`. This is cross-cutting logic that involves both Patron and Ride but since it is a customer/supplier relationship and Ride supplies the tags while Patron expresses the preferences, the logic should live with the Patron.

```elixir
# lib/fun_park/patron.ex
# First we need the machinery for likes/dislikes
def get_likes(%__MODULE__{likes: likes}), do: likes
def get_dislikes(%__MODULE__{dislikes: dislikes}), do: dislikes

def add_likes(%__MODULE__{} = patron, likes)
    when is_list(likes) do
  updated_likes = List.union(likes, get_likes(patron))
  updated_dislikes = List.difference(get_dislikes(patron), updated_likes)

  change(patron, %{
    likes: updated_likes,
    dislikes: updated_dislikes,
  })
end

def remove_likes(%__MODULE__{} = patron, likes)
    when is_list(likes) do
  updated_likes = List.difference(get_likes(patron), likes)
  change(patron, %{likes: updated_likes})
end

def add_dislikes(%__MODULE__{} = patron, dislikes)
    when is_list(dislikes) do
  updated_dislikes = List.union(dislikes, get_dislikes(patron))
  updated_likes = List.difference(get_likes(patron), updated_dislikes)

  change(patron, %{
    dislikes: updated_dislikes,
    likes: updated_likes,
  })
end

def remove_dislikes(%__MODULE__{} = patron, dislikes)
    when is_list(dislikes) do
  updated_dislikes = List.difference(get_dislikes(patron), dislikes)
  change(patron, %{dislikes: updated_dislikes})
end

# Now we need some predicates. We're in the Patron now so we can destructure
# instead of using our new accessors.
def likes_ride?(%__MODULE__{likes: likes}, %Ride{} = ride) do
  Ride.has_any_tag?(ride, likes)
end
```

```elixir
def dislikes_ride?(%__MODULE__{dislikes: dislikes}, %Ride{} = ride) do
  Ride.has_any_tag?(ride, dislikes)
end

# We already have `suggested?/2` to check online/wait time/eligibility.
# We just need to combine it with `likes_ride?/2` and NOT `dislikes_ride?/2`.
def recommended?(%__MODULE__{} = patron, %Ride{} = ride) do
  p_all([
    curry(&likes_ride?/2).(patron),
    p_not(curry(&dislikes_ride?/2).(patron)),
    curry(&Ride.suggested?/2).(patron)
  ]).(ride)
end
```

Chapter 6 - Compose in Context with Monads

- Predicates and monoids translate naturally from their mathematical foundations, but monads are *inspired* by category theory and rather than a direct implementation of a mathematical counterpart. Monads compose computations within a context. In functional programming, *compose* means combining behavior. Monads deal with context in an abstract sense. Context can mean a read-only environment (Reader), the context of absence (Maybe), the context of failure (Either), or the context of asynchronous computation (Effect). We've already been using one monad, the List, where we compose computations within the context of an ordered sequence.

- A Monad has `map` to apply a transformation while preserving structure and `bind` to chain context aware computations.

- Anyone who has mapped over a list has used a Functor. In a List, each item is transformed while the structure stays the same - returning the same number of items in the same order. A Functor follows two rules:

    - Identity - mapping with the identity function returns a copy of the original structure. `map(fn x -> x end, F(a)) = F(a)`
    - Composition - mapping in two steps is the same as mapping once with the composed function{ `map(f, map(g, F(a))) = map(fn x -> f.(g.(x)) end, F(a))`
- Note: It's worth pointing out that `F(a)` above means `a` wrapped in the data structure of the Functor. So when we're dealing with lists, we're only saying mapping identity across the List gives you back the original List.

- These two rules are basically a way to say mapping is nothing but function application, faithfully lifted into the context, with zero extra behavior smuggled in. If the process of mapping changed anything on its own, it wouldn't respect these two rules. List combined with the `Enum.map/2` operation is a Functor. It applies a transformation to each element of a list, preserving the list's structure. The List's length and order are unchanged. Note that `Enum.map/2` does not form a Functor with any enumerable data structure. If you call it on a Map, you get back a List. The easiest way to demonstrate that violates the laws is by mapping Identity and getting back a List value instead of a Map. All Enumerable types collapse to List on return with `Enum.map/2`, so it is only a Functor when used as the operation on Lists.

- A Monad includes behavior for chaining computations within a context, but unlike `map`, there's no universally agreed upon name for this operation, so we will refer to it using Haskell's terminology of `bind`. The `bind` operation respects three laws:

    - Left Identity: wrapping a value then binding it to a function is the same as applying the function directly; `bind(pure(a), f) = f(a)`
    - Right Identity: binding a monad to `pure` has no effect; `bind(m, pure) = m`
    - Associativity: it doesn't matter how you nest your bindings, the result is the same; `bind(bind(m, f), g) = bind(m, fn x -> bind(f(x), g) end)`
- The book hasn't explained `pure` at this point, but it's basically the `wrap` function we defined in previous chapters that puts a value into a minimal context. Left Identity - wrapping a value then binding it to a function gives the same result as just calling the function on the naked value directly; `bind(pure(a), f) = f(a)`. Right Identity - binding an already-wrapped value to pure gives back the same wrapped value, unchanged; bind(m, pure) = m. Since pure does nothing but re-wrap whatever it's handed, unwrapping then immediately re-wrapping is a no-op — pure acts as bind's identity element, the same role empty played for Monoid's append.

- The purpose of the three laws is simply to ensure the chain behaves predictably. Elixir's `Enum` includes `bind` as `flat_map/2`. The basic idea here is "apply a context producing function, then collapse the nested context back down to one level." Again, `Enum.flat_map/2` is a faithful witness for List but not every Enumerable because of its return type. `Enum.flat_map/2` can be thought of as taking a function `a -> [b]`, applying it to each element of a List for a "List of Lists", then flattening it back down to one List of results.

- A function that takes an input and returns a monad is known as a Kleisli function: `kleisli_fn = fn x -> if rem(x, 2) == 0, do: [x * x], else [] end`. The Kleisli

function takes a number and returns a list. Now imagine a list of values `list = [1, 2, 3, 4, 5, 6]`. Then `list |> Enum.flat_map(kleisli_fn)` is `[4, 16, 36]`. Unlike `map`, `bind` allowed us to reshape the structure. We started with a list of 6 elements and ended up with a list of three. `map` and `bind` are both context aware, in this case the context is a list, but `map` transforms each item while preserving the structure and `bind` allows the structure to change.

- **Applicative** is useful when we need to combine two things that are already inside a context. It follow four rules:
  - Identity: Applying a wrapped identity function has no effect. `ap(pure(fn x -> x end), F(a)) = F(a)`
  - Homomorphism: Lifting a function and a value separately is the same as applying them directly. `ap(pure(f), pure(a)) = pure(f.(a))`
  - Interchange: A function in context can be applied to a pure value, or the value can be lifted into a function and applied to the context instead. `ap(F(f), pure(a)) = ap(pure(fn g -> g.(a) end), F(f))`
  - Composition: Applying functions step by step inside the context behaves the same as applying them all at once.

    ```
    compose = fn f -> fn g -> fn x -> f.(g.(x)) end end end
    ap(ap(ap(pure(compose), F(f)), F(g)), F(a)) = ap(F(f), ap(F(g), F(a)))
    ```

- These look complicated, these rules make sure that applying functions in a context behaves just like it would outside the context. There is no Applicative in `Enum` but we can define our own for Lists.

```
ap = fn values, funcs -> for f <- funcs, v <- values, do: f.(v) end
```

- `ap/2` works in the context of lists, taking a list of values and a list of functions and applying each function to every value to produce a new list of results. `ap/2` acts within a context and can reshape the structure. In a series of `bind` each step depends on the result of the previous one; where with `ap` each function is applied independently to each input with no dependency between steps.
- To summarize: `map` transforms; `bind` chains; `ap` collects.

    - Functor's `map`: plain function, wrapped value → wrapped result. `(a -> b) -> F a -> F b`
    - Monad's `bind`: plain function that itself produces a wrapped result, wrapped value → wrapped result (flattened). `(a -> F b) -> F a -> F b`
    - Applicative's `ap`: wrapped function, wrapped value → wrapped result. `F (a -> b) -> F a -> F b`
- Every Monad is an Applicative which itself is a Functor, but not vice-versa. For our purposes, we'll just define a `Monad` protocol that has all three functions. `ap` is derivable from `bind`, as is `map`.

    1. `map/2` applies a function to a value in a context, preserving structure.
    2. `bind/2` sequences computations, allowing each step to determine the next, all within the context.
    3. `ap/2` applies a function to a value where both are in the same context.

```
defprotocol FunPark.Monad do
  def map(monad_value, func)
  def bind(monad_value, func_returning_monad)
  def ap(monadic_func, monad_value)
end
```

- We define `pure/1` with the implementation rather than using protocol dispatch. `pure(5)` is meaningless without knowing what Monad you're wrapping 5 in, and the author chose not to implement a `pure/2` with a witness struct, probably because `pure` is one argument in Haskell.

- The simplest Monad is the `Identity` Monad. It wraps a value without adding any behavior - it's a neutral container that satisfies the monad laws.

```elixir
# lib/fun_park/monad/identity.ex
@enforce_keys [:value]
defstruct [:value]

def pure(value), do: %__MODULE__{value: value}
def extract(%__MODULE__{value: value}), do: value

# Not all Monads implement `Eq`, but many do, including `Identity`.
# `Identity` unwraps its contents and delegates to the `Eq` protocol.
defimpl FunPark.Eq, for: FunPark.Identity do
  alias FunPark.Identity
  alias FunPark.Eq

  def eq?(%Identity{value: v1}, %Identity{value: v2}), do: Eq.eq?(v1, v2)

  def not_eq?(%Identity{value: v1}, %Identity{value: v2}),
    do: Eq.not_eq?(v1, v2)
end

# Identity also implements `Ord`.
defimpl FunPark.Ord, for: FunPark.Identity do
  alias FunPark.Ord
  alias FunPark.Identity

  def lt?(%Identity{value: v1}, %Identity{value: v2}), do: Ord.lt?(v1, v2)
  def le?(%Identity{value: v1}, %Identity{value: v2}), do: Ord.le?(v1, v2)
  def gt?(%Identity{value: v1}, %Identity{value: v2}), do: Ord.gt?(v1, v2)
  def ge?(%Identity{value: v1}, %Identity{value: v2}), do: Ord.ge?(v1, v2)
end

# We need to lift `Eq` and `Ord` to work in our context in order to
# use custom comparators.
def lift_eq(custom_eq) do
  custom_eq = Eq.Utils.to_eq_map(custom_eq)

  %{
    eq?: fn
      %__MODULE__{value: a}, %__MODULE__{value: b} -> custom_eq.eq?.(a, b)
    end,
    not_eq?: fn
      %__MODULE__{value: a}, %__MODULE__{value: b} -> custom_eq.not_eq?(a, b)
    end
  }
end
```

```
  def lift_ord(custom_ord) do
    custom_ord = Ord.Utils.to_ord_map(custom_ord)

    %{
      lt?: fn
        %__MODULE__{value: v1}, %__MODULE__{value: v2} -> custom_ord.lt?.(v1,
v2)
      end,
      le?: fn
        %__MODULE__{value: v1}, %__MODULE__{value: v2} -> custom_ord.le?.(v1,
v2)
      end,
      gt?: fn
        %__MODULE__{value: v1}, %__MODULE__{value: v2} -> custom_ord.gt?.(v1,
v2)
      end,
      ge?: fn
        %__MODULE__{value: v1}, %__MODULE__{value: v2} -> custom_ord.ge?.(v1,
v2)
      end,
    }
  end

  # And of course we implement the Monad protocol.
  defimpl FunPark.Monad, for: FunPark.Identity do
    alias FunPark.Identity

    def map(%Identity{value: value}, func) do
      Identity.pure(func.(value))
    end

    def bind(%Identity{value: value}, func) do
      func.(value)
    end

    def ap(%Identity{value: func}, %Identity{value: value}) do
      Identity.pure(func.(value))
    end
  end
```

- Monads themselves aren't closed, but their core operations are. Functions like `map/2`, `bind/2`, and `ap/2` return a result in the same monadic context, allowing us to pipe them together.

Chapter 7 - Access Shared Environment with Reader

- The Reader Monad represents deferred computation - define now, run later, but with an environment; some shared state or behavior it can read from. Like Identity, it has a single non-branching structure. It adds read-only access to a shared environment. The key to Reader is that it's lazy: it describes the steps for computing a result, but nothing happens until the environment is supplied. It's `bind` allows you to chain several computations that take an environment together while only supplying the environment once.

```elixir
# lib/fun_park/monad/reader.ex
@enforce_keys [:run]
defstruct [:run]

def pure(value), do: %__MODULE__{run: fn _env -> value end}

def run(%__MODULE__{run: f}, env), do: f.(env)
```

- `run` is our escape hatch. It's when we execute what's stored in the Reader with the `env` and get back a value outside the monadic context; much like `unwrap` for our Monoids. Every wrapper type needs an exit function and it's part of what makes it Monad.

```elixir
# lib/fun_park/monad/reader.ex
defimpl FunPark.Monad, for: FunPark.Reader do
  alias FunPark.Reader

  def map(%Reader{run: f}, func),
    do: %Reader{run: fn env -> func.(f.(env)) end}

  def bind(%Reader{run: f}, func),
    do: %Reader{run: fn env -> func.(f.(env)).run.(env) end}

  def ap(%Reader{run: f_func}, %Reader{run: f_value}),
    do: %Reader{run: fn env -> f_func.(env).(f_value.(env)) end}
end
```

- `pure` isn't the only way to get a value inside a Reader — it constructs a minimal Reader whose function just ignores `env`. If that were all we had, we'd never actually use the environment. Other Reader libraries expose `ask/0`, which builds `%Reader{run: fn env -> env end}` — a Reader that just hands back the whole environment. Since Elixir struct fields

aren't private, we can always build one directly, or write a simpler function that pulls out just the parts of env we care about. We're going to define `asks/1` for that.

- The key thing to understand: everything you chain on with `map` / `bind` / `ap` never sees `env` directly — your callbacks only ever receive already-produced values. `env` only gets touched once, at the root: inside whichever Reader you started the chain with (built directly, or via `ask/0` / `asks/1`). From there, Reader's own internal plumbing — the `env ->` closures baked into `map` / `bind` / `ap`'s definitions, not anything you write — is what threads that same `env` down to the root invisibly on your behalf.

```
# lib/fun_park/monad/reader.ex
def asks(func), do: %__MODULE__{run: func}
```

- `asks/1` allows functions within the context to access the read-only copy of the environment. This allows Reader to solve common problems like prop drilling, dependency injection, and shared configuration.
- *prop drilling* is passing information through functions that don't need it and is widely considered an anti-pattern. Let's look at a simple example:

```
iex> alice = FunPark.Patron.make("Alice", 15, 130)
iex> value = 2
iex> square = fn n -> n * n end
iex> message = fn {n, patron} -> "#{patron.name} has #{n}" end
# both functions take one argument, but we still can't compose them because
# one takes a number and the other takes a pair. So we define a helper.
iex> square_tunnel = fn {n, patron} -> {square.(n), patron} end
iex> {value, alice} |> square_tunnel.() |> message.() # "Alice has 4"
# We can do this differently with Reader by defining message/1 to take
# the number and retrieve the patron from the Reader.
iex> reader_message = fn n -> FunPark.Reader.asks(
  fn patron -> "#{patron.name} has #{n}" end
) end
iex> deferred_message = FunPark.Reader.pure(value)
|> FunPark.Monad.map(square)
|> FunPark.Monad.bind(reader_message)
# The Reader monad holds a suspended computation. We use run/2 to resolve it.
iex> FunPark.Reader.run(deferred_message, alice) # "Alice has 4"
# And we could run it again with a different Patron.
iex> beth = FunPark.Patron.make("Beth", 16, 135)
iex> FunPark.Reader.run(deferred_message, beth) # "Beth has 4"
```

- Let's see an example of dependency injection. The essence of dependency injection is the logic stays fixed while the environment varies which isolates domain logic from infrastructure concerns:

```
iex> alice = FunPark.Patron.make("Alice", 15, 130)
iex> prod_service = fn name -> "Hi, #{name}, from prod!" end
iex> test_service = fn name -> "Hi, #{name}, from test!" end
iex> deferred_greeting = fn p -> FunPark.Reader.asks(& &1.(p.name)) end
iex> alice_greeting = deferred_greeting.(alice)
iex> FunPark.Reader.run(alice_greeting, test_service)
# "Hi, Alice, from test!"
iex> FunPark.Reader.run(alice_greeting, prod_service)
# "Hi, Alice, from prod!"
```

- The Reader is also well suited for accessing configuration data such as API keys, feature flags, or endpoint URLs. `config` here is a simple map, but it could just as easily be a database lookup. The ride knows how to apply shared rules but not how they're delivered - preserving a clean boundary between domain behavior and infrastructure.

```
# lib/fun_park/ride.ex
def make_from_env(name) do
  FunPark.Reader.asks(fn config ->
    # We're in FunPark.Ride, so `make` creates a Ride.
    make(name)
    |> change(%{
      min_age: Map.get(config, :min_age, 0),
      min_height: Map.get(config, :min_height, 0),
    })
  end)
end

iex> apple_config = %{min_age: 10, min_height: 120}
# Now let's create a deferred_apple, a Ride that waits for its config
iex> deferred_apple = FunPark.Ride.make_from_env("Apple Cart")
iex> apple = FunPark.Reader.run(deferred_apple, apple_config)
```

- Remember, a Reader is a deferred computation, so within its context `Eq` and `Ord` are not defined. Ordering two Readers doesn't make sense — there's nothing to compare until each is run with an environment and produces an actual value.

- In object-oriented programming; prop drilling, dependency injection, and configuration sharing are all treated as separate concerns, but in functional programming they are all variations of deferring access to a required input and the Reader addresses them with a single abstraction.

Chapter 8 - Manage Absence with Maybe

- The Maybe context has two structures, Just for the presence and Nothing for the absence. Maybe is biased towards Just; it assumes presence and diverts when something is absent. Therefore we define `Maybe.pure/1` as `Just.pure/1`. `just?/1` and `nothing?/1` are *refinement predicates* that are used to check whether we're in the Just or Nothing branch.

```elixir
# lib/fun_park/monad/maybe/just.ex
defmodule FunPark.Monad.Maybe.Just do
  @enforce_keys [:value]
  defstruct [:value]

  def pure(nil), do: raise(ArgumentError, "Cannot wrap nil in a Just")
  def pure(value), do: %__MODULE__{value: value}
end

# lib/fun_park/monad/maybe/nothing.ex
defmodule FunPark.Monad.Maybe.Nothing do
  defstruct []

  def pure, do: %__MODULE__{}
end

# lib/fun_park/monad/maybe.ex
def just(value), do: Just.pure(value)
def nothing, do: Nothing.pure()
def pure(value), do: just(value)

def just?(%Just{}), do: true
def just?(_), do: false
def nothing?(%Nothing{}), do: true
def nothing?(_), do: false
```

- Folding collapses the possibilities down to a single value. But Elixir protocol dispatch is struct based. Maybe isn't a struct. Just and Nothing are the actual structs. So we have to code the two possibilities separately.

```elixir
# lib/fun_park/monad/maybe/just.ex
defimpl FunPark.Foldable, for: FunPark.Monad.Maybe.Just do
  alias FunPark.Monad.Maybe.Just

  def fold_l(%Just{value: value}, just_func, _nothing_func) do
    just_func.(value)
  end

  def fold_r(%Just{} = just, just_func, nothing_func) do
    fold_l(just, just_func, nothing_func)
  end
end

# lib/fun_park/monad/maybe/nothing.ex
defimpl FunPark.Foldable, for: FunPark.Monad.Maybe.Nothing do
  alias FunPark.Monad.Maybe.Nothing

  def fold_l(%Nothing{}, _just_func, nothing_func) do
    nothing_func.()
  end

  def fold_r(%Nothing{} = nothing, just_func, nothing_func) do
    fold_l(nothing, just_func, nothing_func)
  end
end
```

- In the Maybe context, replacing Nothing with a default is common enough to warrant `get_or_else/2`.

```elixir
# lib/fun_park/monad/maybe.ex
def get_or_else(maybe, default) do
  fold_l(maybe, fn value -> value end, fn -> default end)
end

# We can also lift other Monads into the context.
def lift_identity(%Identity{} = identity) do
  case identity do
    %Identity{value: nil} -> nothing()
    %Identity{value: value} -> just(value)
  end
end
```

- A predicate captures a business rule, such as "this patron is a VIP." By lifting that rule into a Maybe we are *making illegal states unrepresentable* because non-VIPs are excluded from the context entirely, so we don't need defensive checks. A lifted predicate refines the context according to a business rule.

```
# lib/fun_park/monad/maybe.ex
def lift_predicate(value, predicate) when is_function(predicate, 1) do
  fold_l(
    fn -> predicate.(value) end,
    fn -> just(value) end,
    fn -> nothing() end,
  )
end
```

- Interops let you drop into FP-mode, chain a few operations, then cut back to idiomatic Elixir. Interop functions will translate nil returns and raised exceptions into our functional abstractions.

```
# lib/fun_park/monad/maybe.ex
def from_nil(nil), do: nothing()
def from_nil(value), do: just(value)

def to_nil(%Nothing{}), do: nil
def to_nil(%Just{value: value}), do: value

# lib/fun_park/ride.ex
def get_fast_pass(%Patron{} = patron, %__MODULE__{} = ride) do
  Enum.find(
    Patron.get_fast_passes(patron),
    &FastPass.valid?(&1, ride),
  )
  |> Maybe.from_nil()
end
```

- Maybe has a concept of `Eq` and `Ord`

```elixir
# lib/fun_park/monad/maybe/just.ex
defimpl FunPark.Eq, for: FunPark.Monad.Maybe.Just do
  alias FunPark.Monad.Maybe.{Just, Nothing}
  alias FunPark.Eq

  def eq?(%Just{value: v1}, %Just{value: v2}), do: Eq.eq?(v1, v2)
  def eq?(%Just{}, %Nothing{}), do: false

  def not_eq?(%Just{value: v1}, %Just{value: v2}),
    do: Eq.not_eq?(v1, v2)
  def not_eq?(%Just{}, %Nothing{}), do: true
end

# lib/fun_park/monad/maybe/nothing.ex
defimpl FunPark.Eq, for: FunPark.Monad.Maybe.Nothing do
  alias FunPark.Monad.Maybe.{Just, Nothing}

  def eq?(%Nothing{}, %Nothing{}), do: true
  def eq?(%Nothing{}, %Just{}), do: false

  def not_eq?(%Nothing{}, %Nothing{}), do: false
  def not_eq?(%Nothing{}, %Just{}), do: true
end

# lib/fun_park/monad/maybe/just.ex
defimpl FunPark.Ord, for: FunPark.Monad.Maybe.Just do
  alias FunPark.Monad.Maybe.{Just, Nothing}
  alias FunPark.Ord

  def lt?(%Just{value: v1}, %Just{value: v2}), do: Ord.lt?(v1, v2)
  def lt?(%Just{}, %Nothing{}), do: false
  def le?(%Just{value: v1}, %Just{value: v2}), do: Ord.le?(v1, v2)
  def le?(%Just{}, %Nothing{}), do: false
  def gt?(%Just{value: v1}, %Just{value: v2}), do: Ord.gt?(v1, v2)
  def gt?(%Just{}, %Nothing{}), do: true
  def ge?(%Just{value: v1}, %Just{value: v2}), do: Ord.ge?(v1, v2)
  def ge?(%Just{}, %Nothing{}), do: true
end

# lib/fun_park/monad/maybe/nothing.ex
defimpl FunPark.Ord, for: FunPark.Monad.Maybe.Nothing do
  alias FunPark.Monad.Maybe.{Just, Nothing}

  def lt?(%Nothing{}, %Nothing{}), do: false
  def lt?(%Nothing{}, %Just{}), do: true
```

```elixir
  def le?(%Nothing{}, %Nothing{}), do: true
  def le?(%Nothing{}, %Just{}), do: true
  def gt?(%Nothing{}, %Nothing{}), do: false
  def gt?(%Nothing{}, %Just{}), do: false
  def ge?(%Nothing{}, %Nothing{}), do: true
  def ge?(%Nothing{}, %Just{}), do: false
end

# We need to lift `Eq` and `Ord` to work in our context in order to
# use custom comparators.
# lib/fun_park/monad/maybe.ex
def lift_eq(custom_eq) do
  custom_eq = Eq.Utils.to_eq_map(custom_eq)

  %{
    eq?: fn
      %Just{value: v1}, %Just{value: v2} -> custom_eq.eq?.(v1, v2)
      %Nothing{}, %Nothing{} -> true
      %Nothing{}, %Just{} -> false
      %Just{}, %Nothing{} -> false
    end,
    not_eq?: fn
      %Just{value: v1}, %Just{value: v2} -> custom_eq.not_eq?.(v1, v2)
      %Nothing{}, %Nothing{} -> false
      %Nothing{}, %Just{} -> true
      %Just{}, %Nothing{} -> true
    end,
  }
end

def lift_ord(custom_ord) do
  custom_ord = Ord.Utils.to_ord_map(custom_ord)

  %{
    lt?: fn
      %Just{value: v1}, %Just{value: v2} -> custom_ord.lt?.(v1, v2)
      %Nothing{}, %Nothing{} -> false
      %Nothing{}, %Just{} -> true
      %Just{}, %Nothing{} -> false
    end,
    le?: fn
      %Just{value: v1}, %Just{value: v2} -> custom_ord.le?.(v1, v2)
      %Nothing{}, %Nothing{} -> true
      %Nothing{}, %Just{} -> true
      %Just{}, %Nothing{} -> false
    end,
```

```
    gt?: fn
      %Just{value: v1}, %Just{value: v2} -> custom_ord.gt?.(v1, v2)
      %Nothing{}, %Nothing{} -> false
      %Nothing{}, %Just{} -> false
      %Just{}, %Nothing{} -> true
    end,
    ge?: fn
      %Just{value: v1}, %Just{value: v2} -> custom_ord.ge?.(v1, v2)
      %Nothing{}, %Nothing{} -> true
      %Nothing{}, %Just{} -> false
      %Just{}, %Nothing{} -> true
    end,
  }
end
```

- Maybe nicely models absence. Nothing acts as explicit absence of a value and Just x holds a real value. Recall our priority queue example from earlier. When max returned the sentinel value, the caller had to know to ignore it. But instead we could use a Maybe, and Nothing implies no one is in the queue.

```
# lib/fun_park/patron.ex
def max_priority_maybe_monoid do
  %Monoid.Max{
    value: Maybe.nothing(), # Nothing is less than all Just values.
    ord: Maybe.lift_ord(ord_by_priority())
  }
end

def highest_priority_maybe(patrons) when is_list(patrons) do
  m_concat(
    max_priority_maybe_monoid(),
    patrons |> Enum.map(&Maybe.pure/1)
  )
end
```

- We've spent all this time talking about Maybe without every implementing the Monad protocol. Since it is only closed under its monadic interface, let's do that.

```elixir
# lib/fun_park/monad/maybe/nothing.ex
defimpl FunPark.Monad, for: FunPark.Monad.Maybe.Nothing do
  alias FunPark.Monad.Maybe.Nothing

  def map(%Nothing{}, _func), do: %Nothing{}
  def ap(%Nothing{}, _val), do: %Nothing{}
  def bind(%Nothing{}, _func), do: %Nothing{}
end

# lib/fun_park/monad/maybe/just.ex
defimpl FunPark.Monad, for: FunPark.Monad.Maybe.Just do
  alias FunPark.Monad.Maybe.{Just, Nothing}

  def map(%Just{value: value}, func), do: Just.pure(func.(value))

  def ap(%Just{value: func}, %Just{value: value}),
    do: Just.pure(func.(value))

  def ap(%Just{}, %Nothing{}), do: %Nothing{}

  def bind(%Just{value: value}, func), do: func.(value)
end
```

- Now we have a new business rule. Wait time should only be updated for online rides. How would we do that with our `online?/1` predicate and Maybe.

```elixir
# lib/fun_park/ride.ex
def update_wait_time_maybe(%__MODULE__{} = ride, wait_time)
    when is_number(wait_time) do

  ride
  |> Maybe.lift_predicate(&online?/1)
  |> map(&update_wait_time(&1, wait_time))
end
```

- Again we make illegal state unrepresentable. Within the `MaybeOnlineRide` context, it's impossible to accidentally update wait time of an offline ride.
- Let's add a fallback function for Nothing. We also want a way to flatten a List of Maybe into a List of values (catMaybes in Haskell).

```elixir
# lib/fun_park/monad/maybe.ex
def or_else(%Nothing{}, fallback_fun) when is_function(fallback_fun, 0),
  do: fallback_fun.()

def or_else(%Just{} = just, _fallback_fun), do: just

def concat(list) when is_list(list) do
  list
  |> fold_l([], fn
    %Just{value: value}, acc -> [value | acc]
    %Nothing{}, acc -> acc
  end)
  |> :lists.reverse()
end

# concat_map takes a list and a kleisli function to Maybe and returns a
# list of values.
def concat_map(list, func) when is_list(list) and is_function(func, 1) do
  fold_l(list, [], fn item, acc ->
    case func.(item) do
      %Just{value: value} -> [value | acc]
      %Nothing{} -> acc
    end
  end)
  |> :lists.reverse()
end
```

- So now let's reimplement our priority queue logic with Maybe.

```elixir
# lib/fun_park/ride.ex
def check_ride_eligibility(%Patron{} = patron, %__MODULE__{} = ride) do
  is_eligible = curry_r(&eligible?/2)
  Maybe.lift_predicate(patron, is_eligible.(ride))
end

def check_fast_pass(%Patron{} = patron, %__MODULE__{} = ride) do
  has_fast_pass = curry_r(&fast_pass?/2)
  Maybe.lift_predicate(patron, has_fast_pass.(ride))
end

def check_vip_or_fast_pass(%Patron{} = patron, %__MODULE__{} = ride) do
  is_vip = &Patron.vip?/1

  patron
  |> Maybe.lift_predicate(is_vip)
  |> Maybe.or_else(fn -> check_fast_pass(patron, ride) end)
end

def fast_pass_lane(%Patron{} = patron, %__MODULE__{} = ride) do
  check_vip_or_pass = curry_r(&check_vip_or_fast_pass/2)

  patron
  |> check_ride_eligibility(ride)
  |> bind(check_vip_or_pass.(ride))
end

def only_fast_pass_lane(patrons, %__MODULE__{} = ride)
    when is_list(patrons) do
  patrons
  |> Maybe.concat_map(&fast_pass_lane(&1, ride))
end
```

- This works well for individuals, but Patrons come in groups. What we want to know is if the entire group can be in the Fast Pass lane or not. What we need is a *sequence*. In the context of Maybe, sequence returns `Just list` if every element in a list is a Just, or `Nothing` if any element is Nothing.

```elixir
# lib/fun_park/monad/maybe.ex
def sequence([]), do: pure([])

def sequence([head | tail]) do
  bind(head, fn value ->
    bind(sequence(tail), fn rest ->
      pure([value | rest])
    end)
  end)
end

# lib/fun_park/ride.ex
def group_fast_pass_lane(patrons, %__MODULE__{} = ride)
    when is_list(patrons) do
  patrons
  |> Enum.map(&fast_pass_lane(&1, ride))
  |> Maybe.sequence()
end
```

- Mapping then sequencing is an operation know as `traverse`. This example works perfectly with a drawback. Since we used `Enum.map/2` and it is eager, this evaluates the whole list instead of short circuiting early. We could write it ourself like we did `sequence/`, but the problem is that isn't tail recursive so we overflow the stack for large lists. Instead we will use Elixir's built in `Enum.reduce_while/3` which is tail recursive and supports short-circuiting.

```elixir
# lib/fun_park/monad/maybe.ex
def traverse([], _func), do: pure([])

def traverse(list, func) when is_list(list) and is_function(func, 1) do
  list
  |> Enum.reduce_while(pure([]), fn item, %Just{value: acc} ->
    case func.(item) do
      %Just{value: value} -> {:cont, pure([value | acc])}
      %Nothing{} -> {:halt, nothing()}
    end
  end)
  |> map(&:lists.reverse/1)
end

# replacing earlier's non tail recursive version
def sequence(list) when is_list(list), do: traverse(list, fn x -> x end)

# lib/fun_park/ride.ex
def group_fast_pass_lane(patrons, %__MODULE__{} = ride)
    when is_list(patrons) do
  Maybe.traverse(patrons, &fast_pass_lane(&1, ride))
end
```

- So how would we re-implement our `priority_fast_lane/2` to bring the next group of priority patrons that is ready to go to the top?

```elixir
# Assuming our List of Patrons is a List of Lists for groups of Patrons.
# lib/fun_park/ride.ex
def priority_fast_lane(patrons, %__MODULE__{} = ride)
    when is_list(patrons) do
  m_concat(
    Patron.max_priority_maybe_monoid(),
    patrons |> Enum.map(&group_fast_pass_lane(&1, ride))
  )
end
```

- Filter is more general than removing items from a list. `Filterable` is about conditionally retaining or discarding values in any context. For a list that means removing elements that don't meet a condition; for Maybe, it means keeping just the value if the condition holds.

```elixir
# lib/fun_park/filterable.ex
defprotocol FunPark.Filterable do
  def guard(structure, bool)
  def filter(structure, predicate)
  def filter_map(structure, func)
end
```

- `Filterable` is three functions:
  - `guard/2` retains the value if the Boolean is true and discards it otherwise.
  - `filter/2` retrains the value if the predicate passes.
  - `filter_map/2` applies a transformation that may also discard the value.

```elixir
# lib/fun_park/monad/maybe/nothing.ex
defimpl FunPark.Filterable, for: FunPark.Monad.Maybe.Nothing do
  alias FunPark.Monad.Maybe.Nothing

  def guard(%Nothing{}, _boolean), do: %Nothing{}
  def filter(%Nothing{}, _predicate), do: %Nothing{}
  def filter_map(%Nothing{}, _func), do: %Nothing{}
end

# lib/fun_park/monad/maybe/just.ex
defimpl FunPark.Filterable, for: FunPark.Monad.Maybe.Just do
  alias FunPark.Monad.Maybe
  alias FunPark.Monad.Maybe.Just
  alias FunPark.Monad

  def guard(%Just{} = maybe, true), do: maybe
  def guard(%Just{}, false), do: Maybe.nothing()

  def filter(%Just{} = maybe, predicate) do
    Monad.bind(maybe, fn value ->
      if predicate.(value) do
        Maybe.pure(value)
      else
        Maybe.nothing()
      end
    end)
  end

  def filter_map(%Just{value: value}, func) do
    case func.(value) do
      %Just{} = just -> just
      _ -> Maybe.nothing()
    end
  end
end
```

- Here's some examples of using the `Filterable` protocol on Maybe. Earlier we updated wait times only for online rides. But what if we also want to make sure wait_time is positive. In the second example, we only want to add a Fast Pass if the Patron is eligible. And we can implement `fast_pass_lane/2` as easily with `filter_map/2` as with `bind/2`. The difference comes down to whether you want to think of it as a sequence of dependent steps (`bind/2`) or more as first check this then continue if it passes (`filter_map/2`).

```elixir
# lib/fun_park/ride.ex
def update_wait_time_maybe(%__MODULE__{} = ride, wait_time)
    when is_number(wait_time) do
  ride
  |> Maybe.lift_predicate(&online?/1)
  |> guard(wait_time >= 0)
  |> map(&update_wait_time(&1, wait_time))
end

# lib/fun_park/patron.ex
def add_fast_pass_maybe(%__MODULE__{} = patron, fast_pass) do
  ride = FastPass.get_ride(fast_pass)
  new_passes = List.union([fast_pass], get_fast_passes(patron))
  update_fast_pass = Utils.curry_r(&change/2)
  eligible = Utils.curry_r(&Ride.eligible?/2)

  patron
  |> Maybe.pure()
  |> filter(eligible.(ride))
  |> map(update_fast_pass.(%{fast_passes: new_passes}))
end

# lib/fun_park/ride.ex
def fast_pass_lane_bind(%Patron{} = patron, %__MODULE__{} = ride) do
  check_vip_or_pass = curry_r(&check_vip_or_fast_pass/2)

  patron
  |> check_ride_eligibility(ride)
  |> bind(check_vip_or_pass.(ride))
end

def fast_pass_lane_filter_map(%Patron{} = patron, %__MODULE__{} = ride) do
  check_vip_or_pass = curry_r(&check_vip_or_fast_pass/2).(ride)

  patron
  |> check_ride_eligibility(ride)
  |> filter_map(check_vip_or_pass)
end
```

Chapter 9 - Model Outcomes with Either

- Either is another branching structure for success or failure, but it adds information to the failure path. Either has Right for success and Left for Failure and both store a value. It

implements `Eq`, `Ord`, `Foldable`, and `Monad`, but unlike Maybe does not implement `Filterable` - it isn't possible to generate a new Left when the predicate fails. The Either Module provides functions for constructing, inspecting, and adapting Either values. Either is biased towards success, which allows `map`, `bind`, and `ap` to compose along the Right branch and short-circuit on Left.

```
# lib/fun_park/monad/either/right.ex
defmodule FunPark.Monad.Either.Right do
  @enforce_keys [:right]
  defstruct [:right]

  def pure(value), do: %__MODULE__{right: value}
end

# lib/fun_park/monad/either/left.ex
defmodule FunPark.Monad.Either.Left do
  @enforce_keys [:left]
  defstruct [:left]

  def pure(value), do: %__MODULE__{left: value}
end
```

- We already used Maybe to model excluding Patrons from the fast lane, but now we want to model that with Either so we can give Patrons a reason. We already have the predicates; we just need to lift them into the Either context.

```
# lib/fun_park/monad/either.ex
def lift_predicate(value, predicate, on_false)
    when is_function(predicate, 1) and is_function(on_false, 1) do
  fold_l(
    fn -> predicate.(value) end,
    fn -> right(value) end,
    fn -> left(on_false.(value)) end
  )
end
```

- A service, as defined by DDD, is a logical grouping of operations that don't naturally belong to a single entity. Since Ride is getting crowded, we are going to introduce a FastLane service. Services are an organizational tool, grouping related behavior under a shared namespace.

```elixir
# lib/fun_park/ride/fast_lane.ex
def ensure_height(%Patron{} = patron, %Ride{} = ride) do
  patron
  |> Either.lift_predicate(
    curry_r(&Ride.tall_enough?/2).(ride),
    fn p -> "#{Patron.get_name(p)} is not tall enough" end
  )
  |> Either.map_left(&ValidationError.new/1)
end

def ensure_age(%Patron{} = patron, %Ride{} = ride) do
  patron
  |> Either.lift_predicate(
    curry_r(&Ride.old_enough?/2).(ride),
    fn p -> "#{Patron.get_name(p)} is not old enough" end
  )
  |> Either.map_left(&ValidationError.new/1)
end
```

- Note that we're wrapping our left values in a `ValidationError` so downstream callers can distinguish validation issues from other errors. Exceptions in Elixir are just structs that implement the Exception protocol.

```elixir
# lib/fun_park/errors/validation_error.ex
defmodule FunPark.Errors.ValidationError do
  defstruct errors: [], __exception__: true

  @behaviour Exception

  def new(errors) when is_list(errors), do: %__MODULE__{errors: errors}

  def new(error), do: %__MODULE__{errors: [error]}

  def merge(%__MODULE__{errors: e1}, %__MODULE__{errors: e2}),
    do: %__MODULE__{errors: e1 ++ e2}

  @impl Exception
  def exception(args) when is_list(args), do: struct(__MODULE__, args)

  @impl Exception
  def exception(message) when is_binary(message),
    do: %__MODULE__{errors: [message]}

  @impl Exception
  def message(%__MODULE__{errors: errors}) do
    Enum.map_join(errors, ", ", &to_string/1)
  end
end
```

- We need to be able to combine rules, so we need `bind`. The book implements it off-screen, but I pulled the code out here before using it. I also pulled the `to_string` definition, since we haven't seen one of those in a while.

```elixir
# lib/fun_park/monad/either/right.ex
defimpl String.Chars, for: FunPark.Monad.Either.Right do
  alias FunPark.Monad.Either.Right

  def to_string(%Right{right: value}), do: "Right(#{value})"
end


defimpl FunPark.Monad, for: FunPark.Monad.Either.Right do
  alias FunPark.Monad.Either.{Left, Right}

  def ap(%Right{right: func}, %Right{right: value}), do: Right.pure(func.
(value))
  def ap(%Right{right: _func}, %Left{} = left), do: left

  def bind(%Right{right: value}, func), do: func.(value)
  def map(%Right{right: value}, func), do: Right.pure(func.(value))
end

# lib/fun_park/monad/either/left.ex
defimpl String.Chars, for: FunPark.Monad.Either.Left do
  alias FunPark.Monad.Either.Left

  def to_string(%Left{left: left}), do: "Left(#{left})"
end


defimpl FunPark.Monad, for: FunPark.Monad.Either.Left do
  alias FunPark.Monad.Either.{Left, Right}
  def map(%Left{} = left, _func), do: left

  def ap(%Left{} = left, %Left{}), do: left
  def ap(%Left{} = left, %Right{}), do: left

  def bind(%Left{} = left, _func), do: left
end

# lib/fun_park/ride/fast_lane.ex
def ensure_eligibility(%Patron{} = patron, %Ride{} = ride) do
  validate_height = curry_r(&ensure_height/2)

  patron
  |> ensure_age(ride)
  |> bind(validate_height.(ride))
end
```

```elixir
def ensure_fast_pass(%Patron{} = patron, %Ride{} = ride) do
  patron
  |> Either.lift_predicate(
    curry_r(&Ride.fast_pass?/2).(ride),
    fn p -> "#{Patron.get_name(p)} does not have a fast pass" end
  )
  |> Either.map_left(&ValidationError.new/1)
end

def ensure_vip_or_fast_pass(%Patron{} = patron, %Ride{} = ride) do
  patron
  |> Either.lift_predicate(
    &Patron.vip?/1,
    fn p -> "#{Patron.get_name(p)} is not a VIP" end
  )
  |> Either.map_left(&ValidationError.new/1)
  |> Either.or_else(fn -> ensure_fast_pass(patron, ride) end)
end
```

- We defined a `map_left/2` that maps a function over left values and ignores any Right values, in addition to `map/2` with works with Right values.
- The last example introduces `or_else/2`. It isn't explicitly mentioned by the book, but is similar to `or_else/2` on Maybe. I've pulled the code out of the repo, along with `get_or_else/2` and `filter_or_else/2` and `right/1`, `left/1`, `right?/1`, and `left?/1`.

```elixir
# lib/fun_park/monad/either.ex
def right(value), do: Right.pure(value)
def left(value), do: Left.pure(value)
def pure(value), do: right(value)

def right?(%Right{}), do: true
def right?(_), do: false

def left?(%Left{}), do: true
def left?(_), do: false

def filter_or_else(either, predicate, left_func) do
  fold_l(
    either,
    fn value ->
      if predicate.(value) do
        either
      else
        left(left_func.())
      end
    end,
    fn _left_value -> either end
  )
end

def get_or_else(either, default) do
  fold_l(
    either,
    fn value -> value end,
    fn _ -> default end
  )
end

def or_else(%Left{}, fallback_fun) when is_function(fallback_fun, 0), do:
fallback_fun.()
def or_else(%Right{} = right, _), do: right

# There's a lot more code in Either.ex, including some unsurprising Eq and
# Ord implementations, a `flip/1`, a `lift_maybe/2` and some list functions
# but the book covers `traverse_a/2` shortly.
# I will explicitly list `from_try/1` and  `from_result/1` since they cover
# translation from other error handling schemes.
def flip(%Left{left: l}), do: %Right{right: l}
def flip(%Right{right: r}), do: %Left{left: r}
```

```
def concat(list) do
  list
  |> fold_l([], fn
    %Right{right: value}, acc -> [value | acc]
    %Left{}, acc -> acc
  end)
  |> :lists.reverse()
end

def concat_map(list, func) do
  list
  |> fold_l([], fn item, acc ->
    case func.(item) do
      %Right{right: value} -> [value | acc]
      %Left{} -> acc
    end
  end)
  |> :lists.reverse()
end

def sequence(list), do: traverse(list, & &1)

def traverse([], _func), do: pure([])

def traverse(list, func) do
  Enum.reduce_while(list, pure([]), fn item, %Right{right: acc} ->
    case func.(item) do
      %Right{right: value} -> {:cont, pure([value | acc])}
      %Left{} = left -> {:halt, left}
    end
  end)
  |> map(&:lists.reverse/1)
end

def from_result({:ok, value}), do: right(value)
def from_result({:error, reason}), do: left(reason)

def to_result(either)
    when is_struct(either, Right) or
           is_struct(either, Left) do
  case either do
    %Right{right: value} -> {:ok, value}
    %Left{left: reason} -> {:error, reason}
  end
end
```

```
def from_try(func) do
  try do
    result = func.()
    right(result)
  rescue
    exception ->
      left(exception)
  end
end

def to_try!(%Right{right: value}), do: value

def to_try!(%Left{left: reason}) do
  raise normalize_reason(reason)
end

defp normalize_reason(%_{} = exception), do: exception

defp normalize_reason(reason) when is_list(reason),
  do: Enum.map_join(reason, ", ", &to_string/1)

defp normalize_reason(reason) when is_binary(reason), do: reason

defp normalize_reason(reason), do: "Unexpected error: #{inspect(reason)}"
```

- Back to domain problems! We need to chain eligibility checks so we don't allow Patrons in the line for a ride they don't qualify for.

```elixir
def ensure_fast_pass_lane(%Patron{} = patron, %Ride{} = ride) do
  ensure_vip_or_pass = curry_r(&ensure_vip_or_fast_pass/2)

  patron
  |> ensure_eligibility(ride)
  |> bind(ensure_vip_or_pass.(ride))
end

def ensure_fast_pass_lane_group(
  patrons,
  %Ride{} = ride,
)
when is_list(patrons) do
  eligible_for_fast_lane = curry_r(&ensure_fast_pass_lane/2)

Either.traverse(
    patrons,
    eligible_for_fast_lane.(ride)
  )
end
```

- Remember what `traverse` does. It takes a list of Either's and converts them into one `Right list` or short-circuits on the first Left.
- And I needed an explanation for why we curry sometimes and not others... Rule of thumb: if a function is called directly inside a pipe stage with all its arguments already at hand, it never needs currying — the pipe supplies the missing piece. But if you're handing a function to `bind` / `map` / `ap` as a value (something they'll call, not you), it needs to already be down to one remaining argument. That's what `curry_r` is for when the function's real arity is higher."
- Monads are useful for dependent steps and for expensive operations where early failure avoids unnecessary work, but sometimes we want to run multiple checks and return all the results instead of the first failure. To solve this we need to switch from monadic traversal to applicative traversal, which runs checks independently and collects the results into an Either - either a Right (list_of_results) or a Left(list_of_reasons).

```elixir
# lib/fun_park/monad/either.ex
def traverse_a([], _func), do: right([])

def traverse_a(list, func) when is_list(list) and is_function(func, 1) do
  fold_l(list, right([]), fn item, acc_result ->
    case {func.(item), acc_result} do
      {%Right{right: value}, %Right{right: acc}} ->
        right([value | acc])

      {%Left{left: new}, %Left{left: existing}} ->
        left(append(existing, coerce(new)))

      {%Right{}, %Left{left: existing}} ->
        left(existing)

      {%Left{left: err}, %Right{}} ->
        left(coerce(err))
    end
  end)
  |> map(&:lists.reverse/1) # Remember: map only modifies Right.
end

# Appendable is a generic interface for combining items of the same kind.
# lib/fun_park/appendable.ex
defprotocol FunPark.Appendable do
  @fallback_to_any true

  def coerce(term)

  def append(accumulator, coerced)
end


defimpl FunPark.Appendable, for: Any do
  def coerce(value) when is_list(value), do: value
  def coerce(value), do: [value]

  def append(acc, value), do: coerce(acc) ++ coerce(value)
end

# And ValidationError implements Appendable:
# lib/fun_park/errors/validation_error.ex
defimpl FunPark.Appendable, for: FunPark.Errors.ValidationError do
  alias FunPark.Errors.ValidationError
```

```
  # If you're working with a ValidationError, e is already a list from `new`.
  def coerce(%ValidationError{errors: e}), do: ValidationError.new(e)

  def append(%ValidationError{} = acc, %ValidationError{} = value) do
    ValidationError.merge(acc, value)
  end
end
```

- Instead of checking a list of Patrons multiple ways, we're going to collect multiple ways to check and apply it to a Patron:

```
iex> valid_height = FunPark.Utils.curry_r(
  &FunPark.Ride.FastLane.ensure_height/2
)
iex> valid_age = FunPark.Utils.curry_r(&FunPark.Ride.FastLane.ensure_age/2)
iex> validators = [
  valid_height.(haunted_mansion),
  valid_age.(haunted_mansion),
]
iex> FunPark.Monad.Either.traverse_a(validators, & &1.(alice))
# Returns a Left of one validation error.
iex> FunPark.Monad.Either.traverse_a(validators, & &1.(beth))
# Also returns a Left of one validation error.
iex> FunPark.Monad.Either.traverse_a(validators, & &1.(charles))
# Also returns a Left of two validation errors.
iex> FunPark.Monad.Either.traverse_a(validators, & &1.(dave))
# Also returns a Right with two Dave's.
```

- So we need a way to apply a list of validations and transform the list of success back to the original value.

```elixir
# lib/fun_park/monad/either.ex
def validate(value, validators) when is_list(validators) do
  traverse_a(validators, fn validator -> validator.(value) end)
  |> map(fn _ -> value end)
end

# lib/fun_park/ride/fast_lane.ex
def validate_eligibility(%Patron{} = patron, %Ride{} = ride) do
  validate_height = curry_r(&ensure_height/2)
  validate_age = curry_r(&ensure_age/2)
  patron
  |> Either.validate([validate_height.(ride), validate_age.(ride)])
end

# lib/fun_park/ride.ex
def ensure_online(%__MODULE__{} = ride) do
  Either.lift_predicate(
    ride,
    &online?/1,
    fn r -> "#{r.name} is offline" end
  )
  |> Either.map_left(&ValidationError.new/1)
end

def validate_fast_pass_lane(%Patron{} = patron, %Ride{} = ride) do
  validate_eligibility = curry(&validate_eligibility/2)
  validate_vip_or_pass = curry(&ensure_vip_or_fast_pass/2)

  Either.validate(
    ride,
    [
      validate_eligibility.(patron),
      validate_vip_or_pass.(patron),
      &Ride.ensure_online/1
    ]
  )
  |> map(fn _ -> patron end)
end

# lib/fun_park/ride/fast_lane.ex
def validate_fast_pass_lane_group(
  patrons,
  %Ride{} = ride,
)
when is_list(patrons) do
```

```
  validate_fast_lane = curry_r(&validate_fast_pass_lane/2)

  # Here we use `traverse_a` on a list of Patrons and a single validation.
  patrons
  |> Either.traverse_a(validate_fast_lane.(ride))
end
```

- The chapter ends with an interesting value of making an in-memory-key-value store for Rides and wrapping the ETS results in Either with simplified error messages instead of crashing on error since ETS throws. After defining basic ETS access functions, it makes a repo. This code is in `lib/fun_park/store.ex` for the ETS functions and `lib/fun_park/ride/repo.ex` for the repository.

Chapter 10 - Coordinate Tasks with Effect

- Elixir developers typically manage I/O using Task, a lightweight abstraction built on Erlang's process model. Task is problematic from a functional perspective for three reasons: it runs eagerly, it breaks the separation between definition and execution, and it raises errors instead of keeping them in the return value. Our Effect class, inspired by Scala's ZIO, uses Either to represent success or failure and Reader to provide a shared environment.

```elixir
# lib/fun_park/monad/effect/right.ex
def pure(value) do
  %__MODULE__{
    effect: fn _env ->
      Task.async(fn -> Either.pure(value) end)
    end
  }
end

# lib/fun_park/monad/effect/left.ex
def pure(value) do
  %__MODULE__{
    effect: fn _env ->
      Task.async(fn -> Either.left(value) end)
    end
  }
end

# lib/fun_park/monad/effect.ex
# `%_{...}` matches a struct of any module — that's what lets `run/2` accept
# either a Right or Left Effect without a separate clause for each
def run(%_{effect: thunk}, env \\ %{}), do: execute_effect(thunk.(env))

defp execute_effect(task) do
  start_time = System.monotonic_time(:millisecond)

  result =
    try do
      case Task.yield(task, 5000) || Task.shutdown(task) do
        {:ok, %Either.Right{} = right} ->
          right

        {:ok, %Either.Left{} = left} ->
          left

        {:ok, other} ->
          Either.left(
            EffectError.new(
              :run,
              {:invalid_result, other}
            )
          )

        nil ->
          Either.left(EffectError.new(:run, :timeout))
```

```
      end
    rescue
      error -> Either.left(EffectError.new(:run, error))
    end

  elapsed = System.monotonic_time(:millisecond) - start_time
  IO.puts("Task completed in #{elapsed}ms")

  result
end
```

- The Effect module constructs values using `right/1`, `left/1`, or `pure/1`. Access to the environment is through `asks/1`. It includes `lift_func/1`, `lift_either/1`, `lift_maybe/2`, and `lift_predicate/3` lifting functions. It has `from_result/1`, `to_result/1`, `from_try/1`, and `to_try!/1` interops functions. And it supports sequencing through `traverse/2`, `sequence/1`, `traverse_a/2`, `sequence_a/1`, and `validate/2`.
- We're going to make a Repo that uses an ETS to store tables about Rides.

```
# lib/fun_park/maintenance/store.ex
def add(%Ride{} = ride, table) do
  Effect.lift_either(fn -> Store.insert_item(table, ride) end)
  |> map(&simulate_delay/1)
  |> Effect.map_left(&simulate_delay/1)
end

def get(%Ride{id: id}, table) do
  Effect.lift_either(fn -> Store.get_item(table, id) end)
  |> map(&simulate_delay/1)
  |> Effect.map_left(&simulate_delay/1)
end

def remove(%Ride{id: id}, table) do
  Effect.lift_either(fn -> Store.delete_item(table, id) end)
  |> map(&simulate_delay/1)
  |> Effect.map_left(&simulate_delay/1)
end

# lib/fun_park/maintenance/repo.ex
def create_store do
  Either.sequence_a([
    Store.create_table(:schedule),
    Store.create_table(:unschedule),
    Store.create_table(:lockout),
    Store.create_table(:compliance),
  ])
end

defp validate_ride_effect(ride) do
  Effect.lift_either(fn -> Ride.validate(ride) end)
end

defp add_to_store_effect(valid_ride) do
  Effect.asks(fn env -> env[:table] end)
  |> bind(fn table -> Store.add(valid_ride, table) end)
end

def add_ride_effect(%Ride{} = ride) do
  validate_ride_effect(ride)
  |> bind(&add_to_store_effect/1)
end

def add_schedule(%Ride{} = ride) do
  ride
```

```
  |> add_ride_effect
  |> Effect.run(%{table: :schedule})
end

# There is also `add_unschedule/1`, `add_lockout/1`, and `add_compliance/1`
# for other tables. And similar `remove_ride_effect/1` function.

# Unlike `add_ride_effect/1` which is tightly coupled to the store, we could
# have a `has_ride_effect/2` where the store is injected.
def has_ride_effect(%Ride{} = ride, table) do
  Effect.asks(fn env -> env[:store] end)
  |> bind(fn store -> store.get(ride, table) end)
  |> map(fn _ -> ride end)
end

def in_schedule(%Ride{} = ride), do: has_ride_effect(ride, :schedule)
# And three others for other tables.
```

- Remember `bind/2` to run Effects sequentially, `traverse_a` to run them all at once.